What Recipe and User Characteristics Influence Users' Reviews of Recipes Online?

Team Members:

- Andy Siu
- Emily Chan
- Danny Espinoza
- Kian M

Project Proposal

This project investigates what recipe and user characteristics influence how users review recipes online. Using the Recipe Reviews and User Feedback dataset from the UC Irvine Machine Learning Repository, our team applied a range of analytical techniques, including K-means clustering, Multinomial Naive Bayes classification, TF-IDF semantic analysis, and Spearman Rank Correlation, to uncover patterns in user behavior, review language, and rating tendencies. Our central question asks whether factors such as user reputation, review engagement (thumbs up/down, reply count), and textual content can predict or explain the star ratings that users assign to recipes.

Presentation Slides: Analyzing User Behavior in Recipe Reviews

Dataset Overview

We use the Recipe Reviews and User Feedback Dataset from the UC Irvine Machine Learning Repository, containing over 18,000 user reviews across 100 recipes. Key features include:

Feature	Description
`recipe_name`	Name of the recipe being reviewed
`user_name` / `user_id`	Reviewer identifiers
`user_reputation`	Numeric score reflecting the reviewer's platform standing
`stars`	Star rating given by the user (0–5)
`thumbs_up` / `thumbs_down`	Community engagement with the review
`reply_count`	Number of replies the review received
`best_score`	Composite quality score for the review
`text`	Full text content of the review

Feature	Description
`created_at`	Unix timestamp of when the review was posted

The dataset arrived in a fairly clean, well-structured format. No significant cleaning was required, save for decoding the review content in the `text` column from HTML entity encoding.

Hypotheses

1. **Veteran reviewers write more balanced reviews:** Reviewers with higher `user_reputation` scores are more likely to produce longer, more nuanced, and balanced reviews, as opposed to simple or extreme ratings.

2. **Extreme reviews attract more engagement:** Reviews that lean toward strongly positive or negative sentiment — in both star rating and language — receive more `thumbs_up` engagement than moderate, balanced reviews.

Analytical Techniques

Technique	Description
K-Means Clustering	Cluster users based on features like reputation, reply count, stars, and review length to identify distinct reviewer profiles/archetypes.
Multinomial Naive Bayes	Classify review text into discrete star ratings (0–5) to evaluate how well language alone predicts a user's rating.
TF-IDF & Cosine Similarity	Analyze keyword frequency across reviews and measure linguistic similarity between recipes to find patterns in review language.
Spearman Rank Correlation	Measure the strength and direction of relationships between user characteristics (e.g. reputation, review length) and review outcomes (e.g. stars, thumbs-up).

In [1]:
```
## General Setup Code

##### Global common code block #####

# This single block will be present in the merged notebook before any other blocks,

# You can think of these as containers for variables and libraries so you can persi

# This one has to be global unfortunately
from types import SimpleNamespace as _Namespace

a = _Namespace() # Andy namespace
d = _Namespace() # Danny namespace
e = _Namespace() # Emily namespace
```

```
k = _Namespace() # Kian namespace

##### The Dataset #####

# The path to the dataset is saved globally, I wrote it under the assumption your p
DATASET_PATH = "../../Dataset/_USE_THIS.csv"

# Can be imported into pandas like:
# k.dataset = k.pd.read_csv(DATASET_PATH)

# Kian: Enable suppression of stdout when not needed
# k.os = __import__("os")
# k.redirect_stdout = __import__("contextlib").redirect_stdout

# Danny - Check if merge here instead of in setupDanny()
# Install NLTK (environment-level, not namespace-level)
import sys
!{sys.executable} -m pip install nltk unidecode
import nltk
required_data = [
    "punkt",
    'punkt_tab',
    "stopwords",
    "wordnet",
    "averaged_perceptron_tagger",
    "vader_lexicon",
    "averaged_perceptron_tagger_eng"
]
for pkg in required_data:
    nltk.download(pkg)

%pip install wordcloud
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: nltk in c:\users\kianm\appdata\local\packages\pythons
oftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\site
-packages (3.9.3)
Requirement already satisfied: unidecode in c:\users\kianm\appdata\local\packages\py
thonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313
\site-packages (1.4.0)
Requirement already satisfied: click in c:\users\kianm\appdata\local\packages\python
softwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\sit
e-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in c:\users\kianm\appdata\local\packages\pytho
nsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\si
te-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in c:\users\kianm\appdata\local\packa
ges\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\pyt
hon313\site-packages (from nltk) (2026.2.28)
Requirement already satisfied: tqdm in c:\users\kianm\appdata\local\packages\pythons
oftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\site
-packages (from nltk) (4.67.3)
Requirement already satisfied: colorama in c:\users\kianm\appdata\local\packages\pyt
honsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313
\site-packages (from click->nltk) (0.4.6)
```

```
[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: C:\Users\kianm\AppData\Local\Microsoft\WindowsApps\PythonSo
ftwareFoundation.Python.3.13_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\kianm\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\kianm\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\kianm\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\kianm\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\kianm\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package vader_lexicon to
[nltk_data]     C:\Users\kianm\AppData\Roaming\nltk_data...
[nltk_data]   Package vader_lexicon is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\kianm\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: wordcloud in c:\users\kianm\appdata\local\packages\py
thonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313
\site-packages (1.9.6)
Requirement already satisfied: numpy>=1.19.3 in c:\users\kianm\appdata\local\package
s\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\pytho
n313\site-packages (from wordcloud) (2.4.1)
Requirement already satisfied: pillow in c:\users\kianm\appdata\local\packages\pytho
nsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\si
te-packages (from wordcloud) (12.1.0)
Requirement already satisfied: matplotlib in c:\users\kianm\appdata\local\packages\p
ythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python31
3\site-packages (from wordcloud) (3.10.8)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\kianm\appdata\local\pack
ages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\py
thon313\site-packages (from matplotlib->wordcloud) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\kianm\appdata\local\packages
\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python
313\site-packages (from matplotlib->wordcloud) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\kianm\appdata\local\pac
kages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\p
ython313\site-packages (from matplotlib->wordcloud) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\kianm\appdata\local\pac
kages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\p
ython313\site-packages (from matplotlib->wordcloud) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\kianm\appdata\local\packa
ges\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\pyt
hon313\site-packages (from matplotlib->wordcloud) (25.0)
Requirement already satisfied: pyparsing>=3 in c:\users\kianm\appdata\local\packages
\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python
313\site-packages (from matplotlib->wordcloud) (3.3.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\kianm\appdata\local
\packages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packag
es\python313\site-packages (from matplotlib->wordcloud) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\kianm\appdata\local\packages\pyt
honsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313
\site-packages (from python-dateutil>=2.7->matplotlib->wordcloud) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

```
[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: C:\Users\kianm\AppData\Local\Microsoft\WindowsApps\PythonSo
ftwareFoundation.Python.3.13_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
```

EDA: User Characteristics vs. Review Outcomes

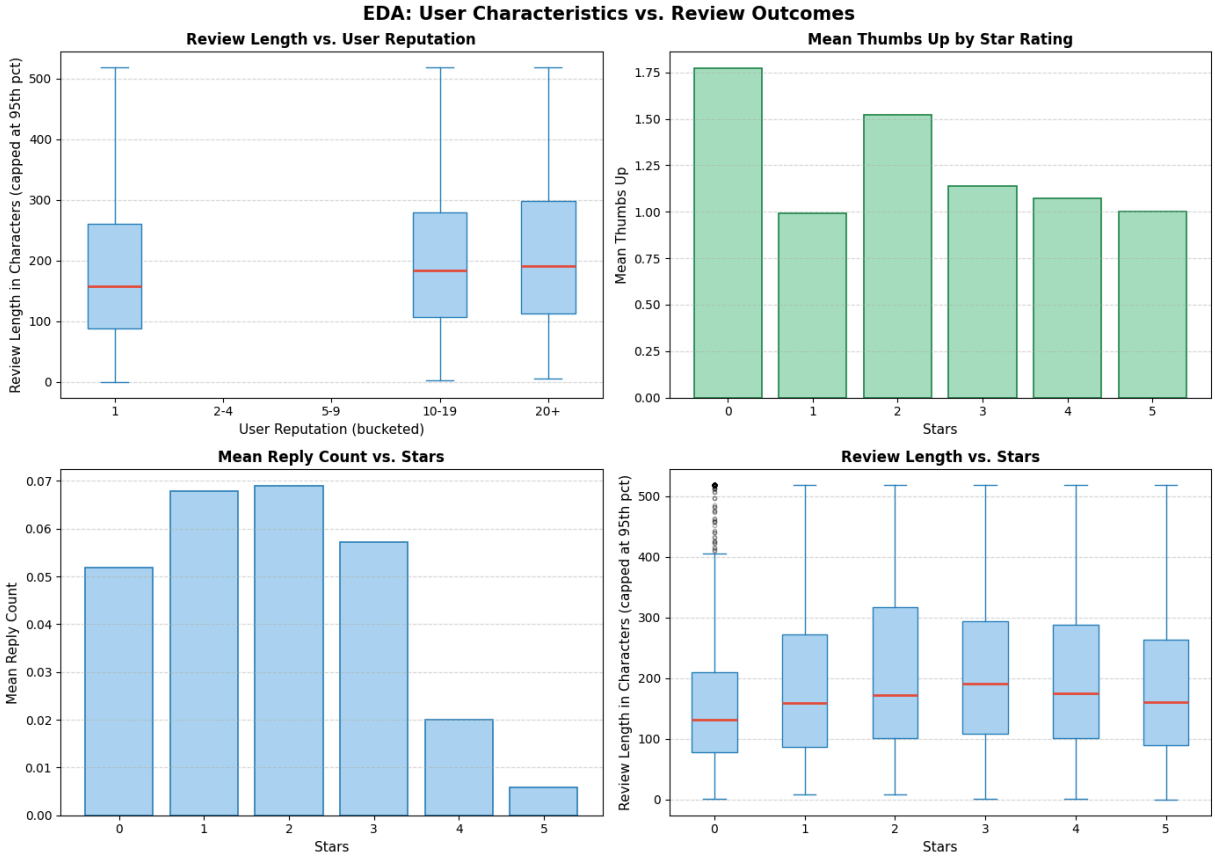
Before running Spearman rank correlation, we first explore the relationships between key user characteristics and review outcomes visually:

- **Review Length vs. User Reputation (bucketed)** — Do higher-reputation (veteran) users write longer reviews?
- **Mean Thumbs Up by Star Rating** — Do extreme reviews (very high or very low stars) attract more upvotes?

- **Mean Reply Count vs. Stars** — Since the reply count is almost always 0, a bar chart of averages shows whether critical reviews spark more discussion.
- **Review Length vs. Stars** — Do certain star ratings correlate with longer or shorter reviews?

In [2]:
```
def andyEDA():
    a.pd = __import__("pandas")
    a.np = __import__("numpy")
    a.plt = __import__("matplotlib.pyplot", fromlist=["pyplot"])

    a.dataset = a.pd.read_csv(DATASET_PATH)
    a.dataset["review_length"] = a.dataset["text"].fillna("").apply(len)
    a.viz_df = a.dataset.copy()

    len_cap = a.viz_df["review_length"].quantile(0.95)
    a.viz_df["review_length_capped"] = a.viz_df["review_length"].clip(upper=len_cap

    a.viz_df["rep_bucket"] = a.pd.cut(
        a.viz_df["user_reputation"],
        bins=[0, 2, 5, 10, 20, 521],
        labels=["1", "2-4", "5-9", "10-19", "20+"],
        right=False
    )

    star_vals = sorted(a.viz_df["stars"].dropna().unique())
    star_labels = [str(int(s)) for s in star_vals]

    box_style = dict(
        patch_artist=True,
        boxprops=dict(facecolor="#AED6F1", color="#2980B9"),
        medianprops=dict(color="#E74C3C", linewidth=2),
        whiskerprops=dict(color="#2980B9"),
        capprops=dict(color="#2980B9"),
        flierprops=dict(marker="o", color="#2980B9", alpha=0.3, markersize=3)
    )

    fig, axes = a.plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle("EDA: User Characteristics vs. Review Outcomes", fontsize=15, font
    axes = axes.flatten()

    # --- Plot 1: Review Length vs. User Reputation buckets (Hypothesis 1) ---
    ax = axes[0]
    rep_buckets = ["1", "2-4", "5-9", "10-19", "20+"]
    groups1 = [a.viz_df[a.viz_df["rep_bucket"] == b]["review_length_capped"].dropna
    ax.boxplot(groups1, tick_labels=rep_buckets, **box_style)
    ax.set_title("Review Length vs. User Reputation", fontsize=12, fontweight="bold
    ax.set_xlabel("User Reputation (bucketed)", fontsize=11)
    ax.set_ylabel("Review Length in Characters (capped at 95th pct)", fontsize=11)
    ax.grid(axis="y", linestyle="--", alpha=0.5)

    # --- Plot 2: Mean Thumbs Up by Star Rating (Hypothesis 2) ---
    ax = axes[1]
    mean_thumbs = a.viz_df.groupby("stars")["thumbs_up"].mean()
    ax.bar([str(int(s)) for s in mean_thumbs.index], mean_thumbs.values,
```

```
            color="#A9DFBF", edgecolor="#1E8449", linewidth=1.2)
    ax.set_title("Mean Thumbs Up by Star Rating", fontsize=12, fontweight="bold")
    ax.set_xlabel("Stars", fontsize=11)
    ax.set_ylabel("Mean Thumbs Up", fontsize=11)
    ax.grid(axis="y", linestyle="--", alpha=0.5)

    # --- Plot 3: Mean Reply Count per Star (bar chart) ---
    ax = axes[2]
    mean_reply = a.viz_df.groupby("stars")["reply_count"].mean()
    ax.bar([str(int(s)) for s in mean_reply.index], mean_reply.values,
            color="#AED6F1", edgecolor="#2980B9", linewidth=1.2)
    ax.set_title("Mean Reply Count vs. Stars", fontsize=12, fontweight="bold")
    ax.set_xlabel("Stars", fontsize=11)
    ax.set_ylabel("Mean Reply Count", fontsize=11)
    ax.grid(axis="y", linestyle="--", alpha=0.5)

    # --- Plot 4: Review Length vs. Stars (box plot) ---
    ax = axes[3]
    groups4 = [a.viz_df[a.viz_df["stars"] == s]["review_length_capped"].dropna() fo
    ax.boxplot(groups4, tick_labels=star_labels, **box_style)
    ax.set_title("Review Length vs. Stars", fontsize=12, fontweight="bold")
    ax.set_xlabel("Stars", fontsize=11)
    ax.set_ylabel("Review Length in Characters (capped at 95th pct)", fontsize=11)
    ax.grid(axis="y", linestyle="--", alpha=0.5)

    a.plt.tight_layout()
    # a.plt.savefig("andy_eda_plots.png", dpi=150, bbox_inches="tight")
    a.plt.show()

andyEDA()
del andyEDA
```



Explanation: Given the 4 distinct graphs, the following is an analysis for each of them:

- **Review Length vs. User Reputation**: Higher-reputation users (buckets 10-19 and 20+) tend to write longer reviews with higher medians and wider spread, suggesting that veteran reviewers do elaborate more. This visually supports Hypothesis 1 and will be tested statistically with Spearman.
- **Mean Thumbs Up by Star Rating**: Reviews at the extremes, particularly 0 and 1 stars, receive more thumbs up on average than mid-range reviews, offering initial support for Hypothesis 2 that extreme reviews attract more engagement.
- **Mean Reply Count vs. Stars**: Reviews with 1 or 2 stars attract the highest average reply count, while 5-star reviews receive almost no replies. Critical reviews clearly spark more discussion than positive ones.
- **Review Length vs. Stars**: Mid-range reviews (2–3 stars) tend to be the longest with the widest spread. 5-star reviews are consistently shorter and more tightly distributed, suggesting that highly positive reviewers write brief comments while mixed-sentiment reviewers elaborate more.

Danny: Data Cleaning

First we must clean the review text for processing with our models. We have to remove any kind of unicode and html errors then lowercase all tokens, and lemmatize all tokens into their base word.

In [3]:
```
def setupDanny():
    d.pd = __import__("pandas")
    d.np = __import__("numpy")
    d.sns = __import__("seaborn")
    d.plt = __import__("matplotlib.pyplot", fromlist=["plt"])
    d.re = __import__("re")
    d.WordCloud = __import__("wordcloud", fromlist=["WordCloud"]).WordCloud

    d.TfidfVectorizer = __import__("sklearn.feature_extraction.text",
                                   fromlist=["TfidfVectorizer"]
                                  ).TfidfVectorizer

    d.cosine_similarity = __import__("sklearn.metrics.pairwise",
                                     fromlist=["cosine_similarity"]
                                    ).cosine_similarity

    # NLTK
    d.nltk = __import__("nltk")
    d.lemmatizer = d.nltk.stem.WordNetLemmatizer()
    d.stopwords = set(d.nltk.corpus.stopwords.words("english"))
    d.stopwords.discard("more")
    d.SentimentIntensityAnalyzer = __import__("nltk.sentiment",
                                              fromlist=["SentimentIntensityAnalyzer
                                             ).SentimentIntensityAnalyzer
    d.vader = d.SentimentIntensityAnalyzer()
    d.unidecode = __import__("unidecode").unidecode

    d.df = d.pd.read_csv(DATASET_PATH)

setupDanny()
del setupDanny
```

In [4]:
```
def cleanData(text):
    # Remove HTML & Unicode & Tokenize
    text = str(text)
    text = d.re.sub(r'<.*?>', ' ', text)
    text = d.unidecode(text).lower()
    tokens = d.nltk.word_tokenize(text)

    # POS Tag all tokens
    pos_tags = d.nltk.pos_tag(tokens)

    cleaned_tokens = []

    # Remove stopwords & Lemmatize Tokens
    for word, tag in pos_tags:
        if word in d.stopwords or not word.isalpha():
            continue
        # Tag into adj, adverb, noun, verb
        wtag = tag[0].lower()
```

```
        wtag = wtag if wtag in ['a', 'r', 'n', 'v'] else 'n'

        # Lemmatize and append to cleaned_tokens
        lemma = d.lemmatizer.lemmatize(word, pos=wtag)
        cleaned_tokens.append(lemma)

    return cleaned_tokens

d.cleanData = cleanData
del cleanData

d.df["tokens"] = d.df["text"].apply(d.cleanData)
d.df["clean_text"] = d.df["tokens"].apply(lambda toks: " ".join(toks))
d.df.loc[:,'text':'clean_text']
```

Out[4]:

	text	tokens	clean_text
0	I tweaked it a little, removed onions because ...	[tweak, little, remove, onion, onion, hater, h...	tweak little remove onion onion hater house us...
1	Bush used to have a white chili bean and it ma...	[bush, use, white, chili, bean, make, recipe, ...	bush use white chili bean make recipe super si...
2	I have a very complicated white chicken chili ...	[complicated, white, chicken, chili, recipe, m...	complicated white chicken chili recipe make ye...
3	In your introduction, you mentioned cream chee...	[introduction, mention, cream, cheese, none, l...	introduction mention cream cheese none list in...
4	Wonderful! I made this for a "Chili/Stew" nigh...	[wonderful, make, night, dinner, church, every...	wonderful make night dinner church everyone wa...
...	...	...	...
18177	This Strawberry Cake has been a family favorit...	[strawberry, cake, family, favorite, since, pa...	strawberry cake family favorite since past sev...
18178	<p>I received endless compliments on this cake...	[receive, endless, compliment, cake, use, firm...	receive endless compliment cake use firmer str...
18179	This cake was delicious and so moist! I didn't...	[cake, delicious, moist, need, sugar, call, fr...	cake delicious moist need sugar call frosting ...
18180	I just made this too. It is wonderful. As fo...	[make, wonderful, frosting, wish, make, thicke...	make wonderful frosting wish make thicker tad ...
18181	OMG, you must make this cake. I made it for my...	[omg, must, make, cake, make, supplant, previo...	omg must make cake make supplant previous favo...

18182 rows × 3 columns

In [5]:

```
def analyzeTokens():
    all_tokens = []
    for token_list in d.df["tokens"]:
        all_tokens.extend(token_list)
```

```
    d.all_tokens = all_tokens
    d.text_obj = d.nltk.Text(all_tokens)
    d.freqdist = d.nltk.FreqDist(all_tokens)

d.analyzeTokens = analyzeTokens
del analyzeTokens
d.analyzeTokens()
```

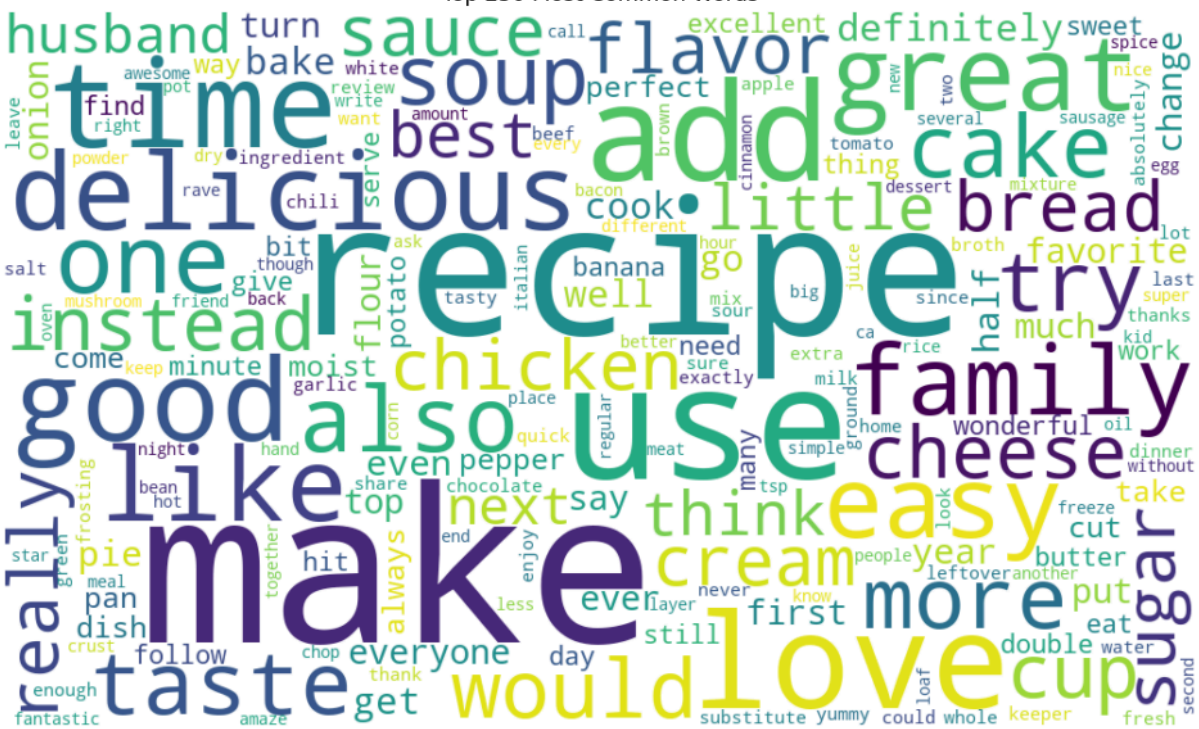
EDA: Word Cloud

What are the most common 250 words across all reviews?

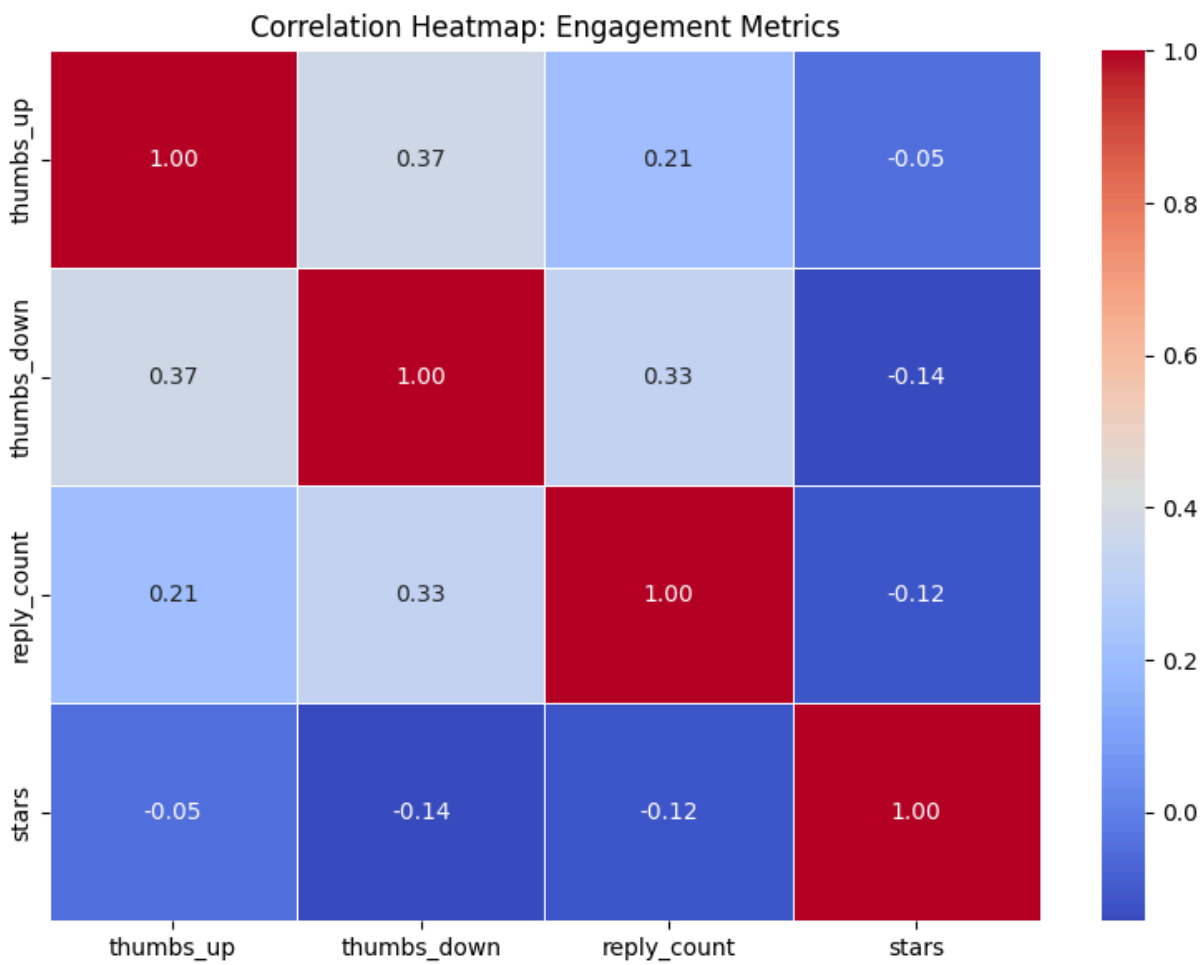
First we wanted to analyze the most common 250 words used throught all the reviews. So we created a word cloud where the bigger a word is the most frequencies we have.
This form of visualization is very common to showcase such a range of diction. So after some visualization we found that words such as: make, recipe, use, add, love, time, good, delicious, easy, family. Are all high frequency words. The first ingredient used is cheese, followed by chicken.

In [6]:
```
def plotWordCloud():
    top250 = dict(d.freqdist.most_common(250))

    wc = d.WordCloud(
        width=1000,
        height=600,
        background_color="white",
        max_font_size=180
    ).generate_from_frequencies(top250)

    d.plt.figure(figsize=(12, 8))
    d.plt.imshow(wc, interpolation='bilinear')
    d.plt.axis("off")
    d.plt.title("Top 250 Most Common Words")
    d.plt.show()

d.plotWordCloud = plotWordCloud
del plotWordCloud
d.plotWordCloud()
```

Top 250 Most Common Words


make
recipe
use
love
time
delicious
add
great
family
easy
cheese
sugar
cake
bread
soup
sauce
flavor
chicken
good
also
like
really
taste
would
cup
more
cream
think
instead
one
try
husband
definitely
excellent
perfect
best
little
cook
banana
well
favorite
much
half
butter
year
first
everyone
get
day
ever
still
put
eat
double
water
pie
pan
dish
follow
hit
always
top
next
say
even
pepper
chocolate
share
meat
simple
many
tsp
take
cut
freeze
look
leftover
another
people
never
know
layer
less
enjoy
end
chop
thank
crust
enough
fantastic
amaze
substitute
yummy
could
whole
keeper
fresh
second
loaf
wonderful
oil
dinner
without
round
go
home
place
quick
regular
corn
garlic
minute
moist
keep
come
need
sure
exactly
milk
extra
rice
work
kid
thanks
super
last
lot
broth
hour
juice
big
since
ca
sour
mix
better
tasty
italian
potato
flour
serve
give
back
friend
mushroom
though
bit
ask
salt
oven
frosting
green
go
star
meal
together
bean
hot
night
hand
bacon
different
dessert
mixture
tomato
thing
several
sausage
absolutely
egg
cinnamon
brown
beef
every
amount
write
want
review
white
apple
new
two
nice
spice
sweet
change
call
turn
bake
way
awesome
pot
find
right
leave
onion
powder
dry
ingredient
rave
chili
serve
potato
flour
corn
italian
tasty
half
since
big
juice
broth
hour

```
    # Plot heatmap
    e.plt.figure(figsize=(8, 6))
    e.sns.heatmap(
        e.correlation_matrix,
        annot=True,
        cmap='coolwarm',
        fmt='.2f',
        linewidths=0.5
    )
    e.plt.title('Correlation Heatmap: Engagement Metrics')
    e.plt.tight_layout()
    e.plt.show()

e.emilyCorrelationHeatmap = emilyCorrelationHeatmap
del emilyCorrelationHeatmap
e.emilyCorrelationHeatmap()
```



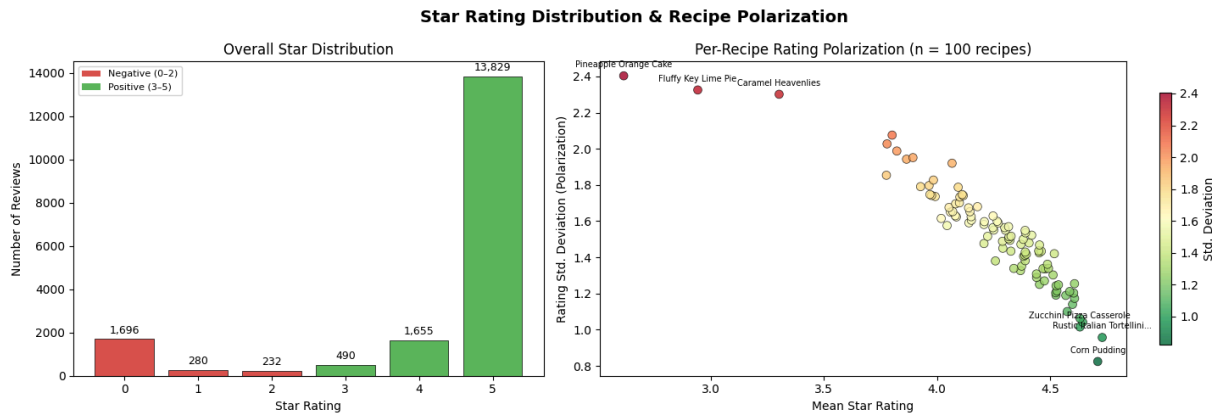
Explanation: This heatmap shows the relationship between four features of a user's review: thumbs_up, thumbs_down, reply_count, and stars. We wanted to see if there was any relationship between these features that influence others' reactions to a review. Overall, there isn't much of a correlation between these features except for a slight relationship between thumbs_down, thumbs_up, and reply count which can indicate a controversial review.

EDA: Star Rating Distribution & Recipe Polarization

Since our Multinomial Naive Bayes classifier later in this report maps star ratings into binary classes (negative vs. positive), it is important to first understand the shape of the rating distribution and how it varies across recipes.

In [9]:
```
def kianEDA():
    k.pd = __import__("pandas")
    k.np = __import__("numpy")
    k.plt = __import__("matplotlib.pyplot", fromlist=["pyplot"])
    k.sns = __import__("seaborn")

    k.df_eda = k.pd.read_csv(DATASET_PATH)

    fig, axes = k.plt.subplots(1, 2, figsize=(15, 5), gridspec_kw={"width_ratios":
    fig.suptitle("Star Rating Distribution & Recipe Polarization", fontsize=14, fon

    # --- Left panel: overall star distribution with binary class coloring ---
    star_counts = k.df_eda["stars"].value_counts().sort_index()
    colors = ["#d9534f" if s <= 2 else "#5cb85c" for s in star_counts.index]
    bars = axes[0].bar(star_counts.index, star_counts.values, color=colors, edgecol

    for bar, count in zip(bars, star_counts.values):
        axes[0].text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 150,
                     f"{count:,}", ha="center", va="bottom", fontsize=9)

    axes[0].set_xlabel("Star Rating")
    axes[0].set_ylabel("Number of Reviews")
    axes[0].set_title("Overall Star Distribution")
    axes[0].set_xticks(range(6))

    from matplotlib.patches import Patch
    axes[0].legend(handles=[Patch(facecolor="#d9534f", label="Negative (0–2)"),
                            Patch(facecolor="#5cb85c", label="Positive (3–5)")],
                   loc="upper left", fontsize=8)

    # --- Right panel: per-recipe rating std (polarization) ---
    recipe_stats = k.df_eda.groupby("recipe_name")["stars"].agg(["mean", "std"]).re
    recipe_stats = recipe_stats.sort_values("std", ascending=True).reset_index(drop

    scatter = axes[1].scatter(recipe_stats["mean"], recipe_stats["std"],
                              c=recipe_stats["std"], cmap="RdYlGn_r",
                              edgecolors="black", linewidth=0.5, s=50, alpha=0.8)

    # Label the most polarized and most consistent
    most_polarized = recipe_stats.nlargest(3, "std")
    most_consistent = recipe_stats.nsmallest(3, "std")
    for _, row in k.pd.concat([most_polarized, most_consistent]).iterrows():
        short_name = row["recipe_name"][:25] + ("..." if len(row["recipe_name"]) >
        axes[1].annotate(short_name, (row["mean"], row["std"]),
                         fontsize=7, ha="center", va="bottom",
                         xytext=(0, 6), textcoords="offset points")
```

```
    axes[1].set_xlabel("Mean Star Rating")
    axes[1].set_ylabel("Rating Std. Deviation (Polarization)")
    axes[1].set_title("Per-Recipe Rating Polarization (n = 100 recipes)")
    k.plt.colorbar(scatter, ax=axes[1], label="Std. Deviation", shrink=0.8)

    k.plt.tight_layout()
    k.plt.show()

    del k.df_eda

kianEDA()
del kianEDA
```



How are star ratings distributed overall?

Do all recipes receive similar ratings, or are some far more polarizing than others?

Spearman Rank Correlation

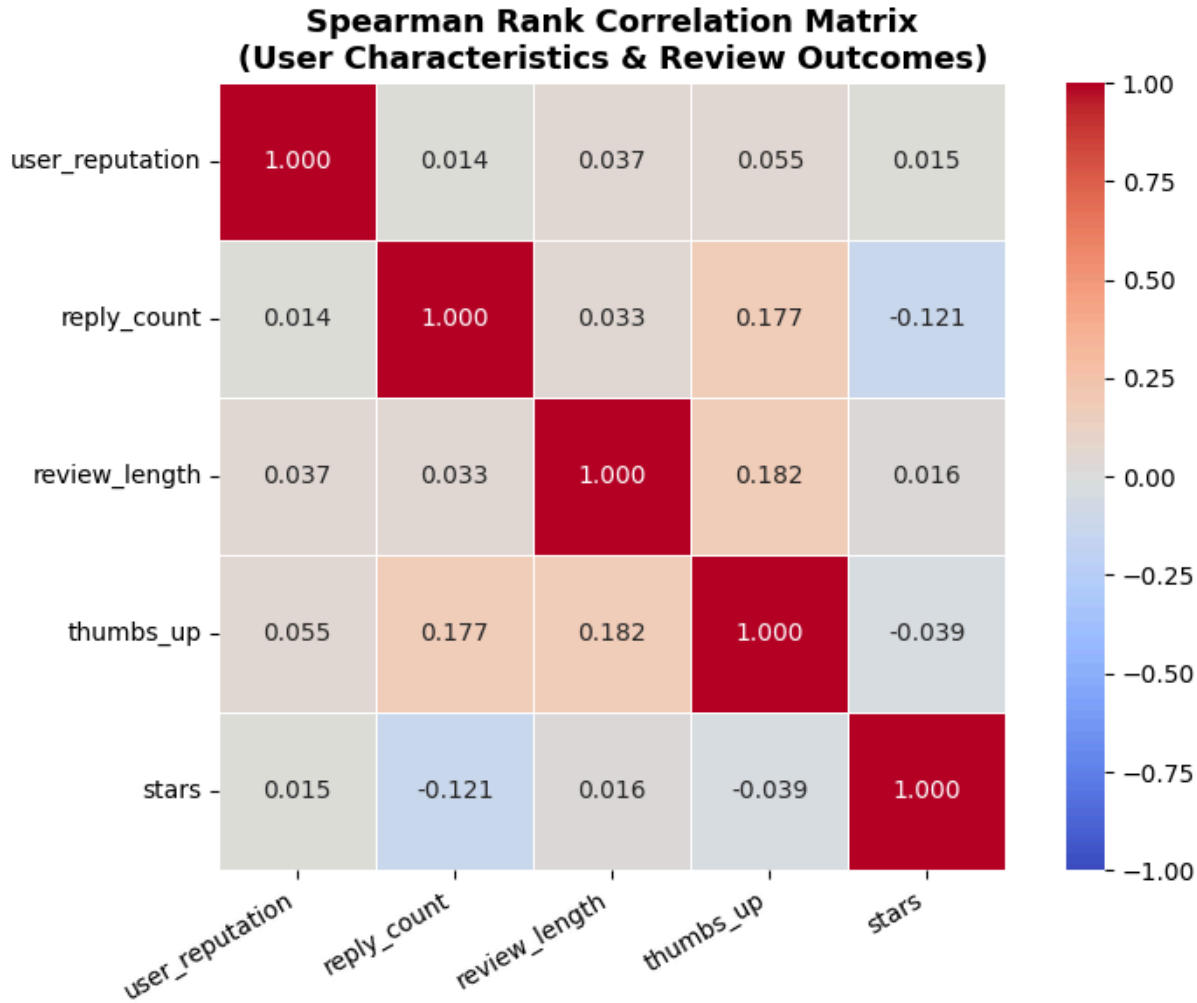
Variable pairs tested:

- `user_reputation` ↔ `stars`
- `user_reputation` ↔ `thumbs_up`
- `user_reputation` ↔ `review_length`
- `reply_count` ↔ `stars`
- `reply_count` ↔ `thumbs_up`
- `review_length` ↔ `stars`
- `review_length` ↔ `thumbs_up`

In [10]:
```
def andySpearman():
    a.spearmanr = __import__("scipy.stats", fromlist=["spearmanr"]).spearmanr
    a.sns = __import__("seaborn")

    if not hasattr(a, "dataset"):
        a.pd = __import__("pandas")
        a.plt = __import__("matplotlib.pyplot", fromlist=["pyplot"])
```

```
        a.np = __import__("numpy")
        a.dataset = a.pd.read_csv(DATASET_PATH)
        a.dataset["review_length"] = a.dataset["text"].fillna("").apply(len)

    df = a.dataset[["user_reputation", "reply_count", "thumbs_up", "review_length",

    # Define pairs to test
    pairs = [
        ("user_reputation", "stars"),
        ("user_reputation", "thumbs_up"),
        ("user_reputation", "review_length"),
        ("reply_count",     "stars"),
        ("reply_count",     "thumbs_up"),
        ("review_length",   "stars"),
        ("review_length",   "thumbs_up"),
    ]

    results = []
    for x, y in pairs:
        rho, pval = a.spearmanr(df[x], df[y])
        results.append({"Variable A": x, "Variable B": y, "rho": round(rho, 4), "p-

    a.spearman_results = a.pd.DataFrame(results)
    a.spearman_results["Significant (p<0.05)"] = a.spearman_results["p-value"] < 0.
    print(a.spearman_results.to_string(index=False))

    cols = ["user_reputation", "reply_count", "review_length", "thumbs_up", "stars"
    rho_matrix = a.pd.DataFrame(index=cols, columns=cols, dtype=float)
    for c1 in cols:
        for c2 in cols:
            rho, _ = a.spearmanr(df[c1], df[c2])
            rho_matrix.loc[c1, c2] = round(rho, 3)

    a.plt.figure(figsize=(8, 6))
    a.sns.heatmap(
        rho_matrix.astype(float),
        annot=True, fmt=".3f",
        cmap="coolwarm", center=0,
        vmin=-1, vmax=1,
        linewidths=0.5,
        square=True
    )
    a.plt.title("Spearman Rank Correlation Matrix\n(User Characteristics & Review O
                fontsize=13, fontweight="bold")
    a.plt.xticks(rotation=30, ha="right")
    a.plt.tight_layout()
    # a.plt.savefig("andy_spearman_heatmap.png", dpi=150, bbox_inches="tight")
    a.plt.show()

andySpearman()
del andySpearman
```

Variable A	Variable B	rho	p-value	Significant (p<0.05)
user_reputation	stars	0.0148	0.045756	True
user_reputation	thumbs_up	0.0547	0.000000	True
user_reputation	review_length	0.0374	0.000000	True
reply_count	stars	-0.1211	0.000000	True
reply_count	thumbs_up	0.1774	0.000000	True
review_length	stars	0.0165	0.026184	True
review_length	thumbs_up	0.1824	0.000000	True



Explanation: Given the results, the following is the interpretation of the Spearman Analysis:

- `User Reputation ↔ Stars (ρ = 0.015)` : Reputation has almost no relationship with star rating.
- `User Reputation ↔ Thumbs Up (ρ = 0.055)` : Reputation to almost no relationship with upvotes.
- `User Reputation ↔ Review Length (ρ = 0.037)` : Veteran users don't really write longer reviews.
- `Reply Count ↔ Stars (ρ = -0.121)` : Lower star reviews get more replies.
- `Reply Count ↔ Thumbs Up (ρ = 0.177)` : Reviews that get replies also tend to get upvotes, suggesting engaged reviews attrack all types of interactions.

- `Review Length ↔ Stars (ρ = 0.016)` : Longer reviews has almost no relationship with star rating.
- `Review Length ↔ Thumbs Up (ρ = 0.181)` : Longer reviews tend to get more upvotes.

Overall conclusion: The correlations are statistically significant but practically weak. No single user characteristic strongly predict review outcomes.

Limitations & Context: The consistently weak correlations across all pairs likely reflect the passive nature of recipe review platforms. The majority of reviews in this dataset have zero thumbs up, zero replies, and a user reputation of 1, indicating that most reviewers are casual, one-time visitors rather than engaged community members. With so little variation in engagement metrics, even meaningful behavioral differences would be difficult to detect statistically. These results are therefore not a failure of the analysis, but rather a finding about the platform itself: user characteristics as measured here are not strong drivers of review outcomes on this site.

K-Means Clustering

Aggregation

In [11]:
```
def emilyUserFeatures():
    # Compute review length
    e.df["review_length"] = e.df["text"].astype(str).str.len()

    # Aggregate user-level features
    e.user_features = (
        e.df.groupby("user_id").agg({
            "user_reputation": "mean",
            "reply_count": "sum",
            "stars": "mean",
            "review_length": "mean"
        }).reset_index()
    )

    # Display first few rows
    print("\nAggregated user features:")
    display(e.user_features.head())

e.emilyUserFeatures = emilyUserFeatures
del emilyUserFeatures
e.emilyUserFeatures()
```

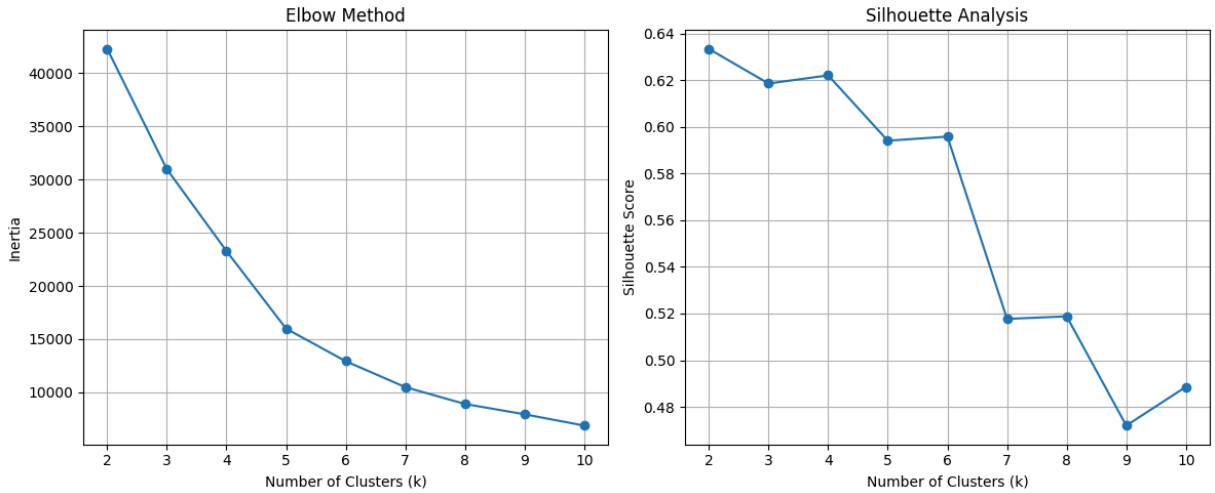
```
Aggregated user features:
```

	user_id	user_reputation	reply_count	stars	review_length
0	u_05PZUpOV27Pv	1.0	0	5.0	191.0
1	u_09Pspx0F3ZKy	1.0	0	5.0	119.0
2	u_0BYS3gNJ4rI0	1.0	0	5.0	95.0
3	u_0GfixeKJgmAL	1.0	0	5.0	41.0
4	u_0HraB0BMR3qu	1.0	0	0.0	39.0

Finding The Optimal K

In [12]:
```
def emilyClusterEvaluation():
    # Feature set
    e.features = ['user_reputation', 'reply_count', 'stars', 'review_length']
    e.X = e.user_features[e.features]

    # Scale features
    e.scaler = e.StandardScaler()
    e.X_scaled = e.scaler.fit_transform(e.X)

    # Evaluate KMeans across k = 2..10
    e.inertia = []
    e.silhouette_scores = []
    e.k_range = range(2, 11)

    for k in e.k_range:
        kmeans = e.KMeans(n_clusters=k, random_state=42, n_init=10)
        kmeans.fit(e.X_scaled)
        e.inertia.append(kmeans.inertia_)
        e.silhouette_scores.append(
            e.silhouette_score(e.X_scaled, kmeans.labels_)
        )

    # Plot elbow + silhouette
    e.plt.figure(figsize=(12, 5))

    # Elbow plot
    e.plt.subplot(1, 2, 1)
    e.plt.plot(e.k_range, e.inertia, 'o-')
    e.plt.xlabel('Number of Clusters (k)')
    e.plt.ylabel('Inertia')
    e.plt.title('Elbow Method')
    e.plt.grid(True)

    # Silhouette plot
    e.plt.subplot(1, 2, 2)
    e.plt.plot(e.k_range, e.silhouette_scores, 'o-')
    e.plt.xlabel('Number of Clusters (k)')
    e.plt.ylabel('Silhouette Score')
    e.plt.title('Silhouette Analysis')
    e.plt.grid(True)
```

```
    e.plt.tight_layout()
    e.plt.show()

e.emilyClusterEvaluation = emilyClusterEvaluation
del emilyClusterEvaluation
e.emilyClusterEvaluation()
```



K-Clustering

In [13]:
```
def emilyClusteringAndPCA():
    # Optimal cluster count
    e.optimal_k = 5
    e.kmeans = e.KMeans(n_clusters=e.optimal_k, random_state=42, n_init=10)

    # Assign clusters
    e.user_features["cluster"] = e.kmeans.fit_predict(e.X_scaled)

    # Cluster analysis table
    e.cluster_analysis = (
        e.user_features.groupby("cluster").agg({
            "user_reputation": ["mean", "std"],
            "reply_count": ["mean", "std"],
            "stars": ["mean", "std"],
            "review_length": ["mean", "std"],
            "user_id": "count"
        })
    )

    print("\nCluster Analysis:")
    display(e.cluster_analysis)

    # Pairplot of features by cluster
    e.plt.figure(figsize=(12, 10))
    e.sns.pairplot(
        e.user_features,
        vars=e.features,
        hue="cluster",
        palette="viridis"
    )
```

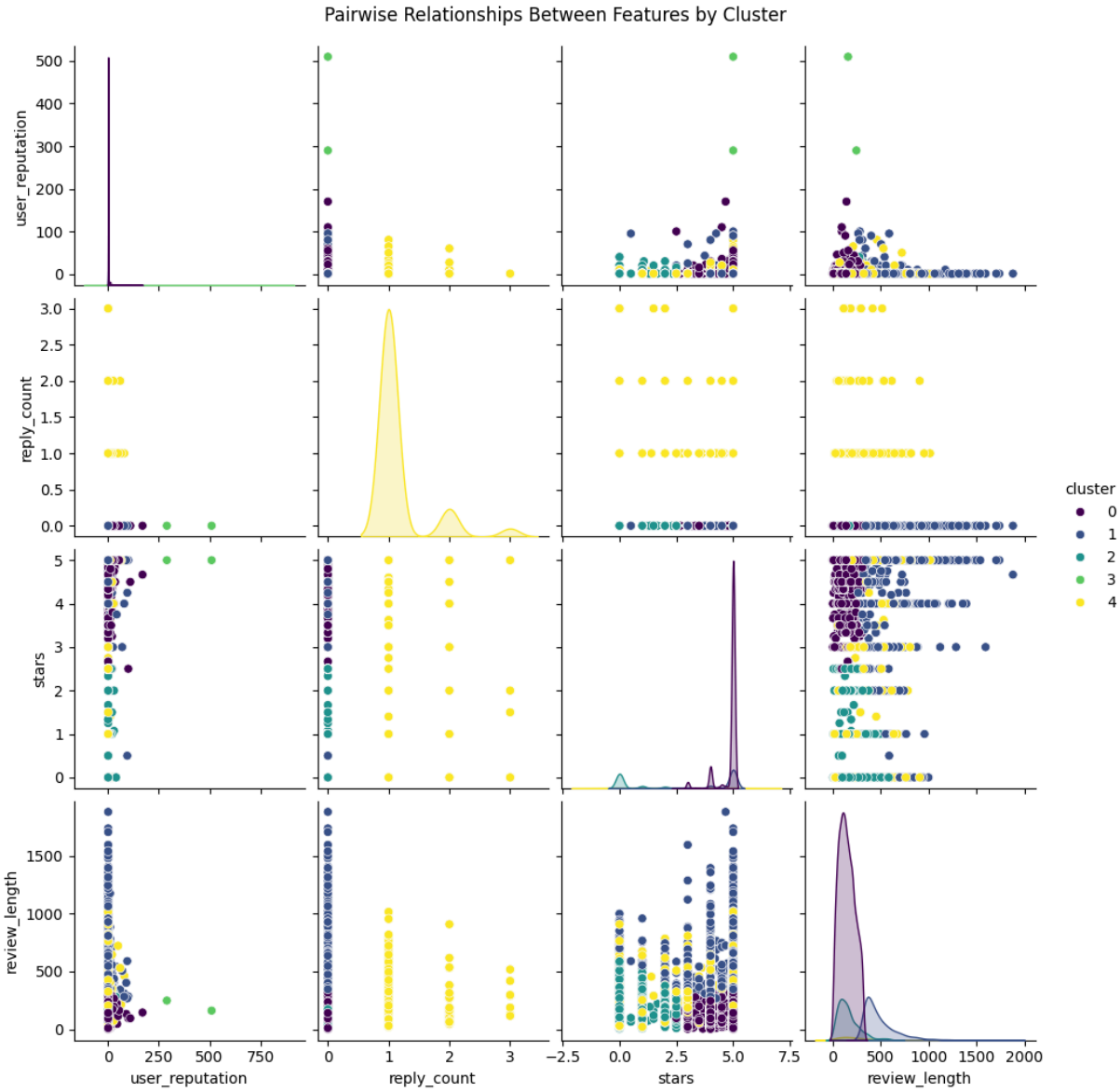
```
e.plt.suptitle("Pairwise Relationships Between Features by Cluster", y=1.02)
e.plt.show()

# PCA for visualization
e.pca = e.PCA(n_components=2)
e.pca_result = e.pca.fit_transform(e.X_scaled)

e.user_features["pca_1"] = e.pca_result[:, 0]
e.user_features["pca_2"] = e.pca_result[:, 1]

# PCA scatterplot
e.plt.figure(figsize=(10, 8))
e.sns.scatterplot(
    x="pca_1",
    y="pca_2",
    hue="cluster",
    data=e.user_features,
    palette="viridis",
    s=100
)
e.plt.title("User Clusters Visualized with PCA")
e.plt.xlabel(f"Principal Component 1 (Variance Explained: {e.pca.explained_vari
e.plt.ylabel(f"Principal Component 2 (Variance Explained: {e.pca.explained_vari
e.plt.show()

# Radar chart function stored in namespace
def radar_chart(cluster_centers, feature_names):
    categories = feature_names
    N = len(categories)

    angles = [n / float(N) * 2 * e.np.pi for n in range(N)]
    angles += angles[:1]

    fig, ax = e.plt.subplots(figsize=(10, 10), subplot_kw=dict(polar=True))
    e.plt.xticks(angles[:-1], categories, size=12)

    for i, center in enumerate(cluster_centers):
        values = center.tolist()
        values += values[:1]
        ax.plot(angles, values, linewidth=2, linestyle="solid", label=f"Cluster
        ax.fill(angles, values, alpha=0.1)

    e.plt.legend(loc="upper right", bbox_to_anchor=(0.1, 0.1))
    e.plt.title("Cluster Profiles", size=15)
    e.plt.show()

e.radar_chart = radar_chart

# Cluster centers
e.cluster_centers = e.kmeans.cluster_centers_
e.cluster_centers_original = e.scaler.inverse_transform(e.cluster_centers)

# Normalize cluster centers for radar chart
e.min_max_scaler = e.StandardScaler()
e.cluster_centers_normalized = e.min_max_scaler.fit_transform(e.cluster_centers
```

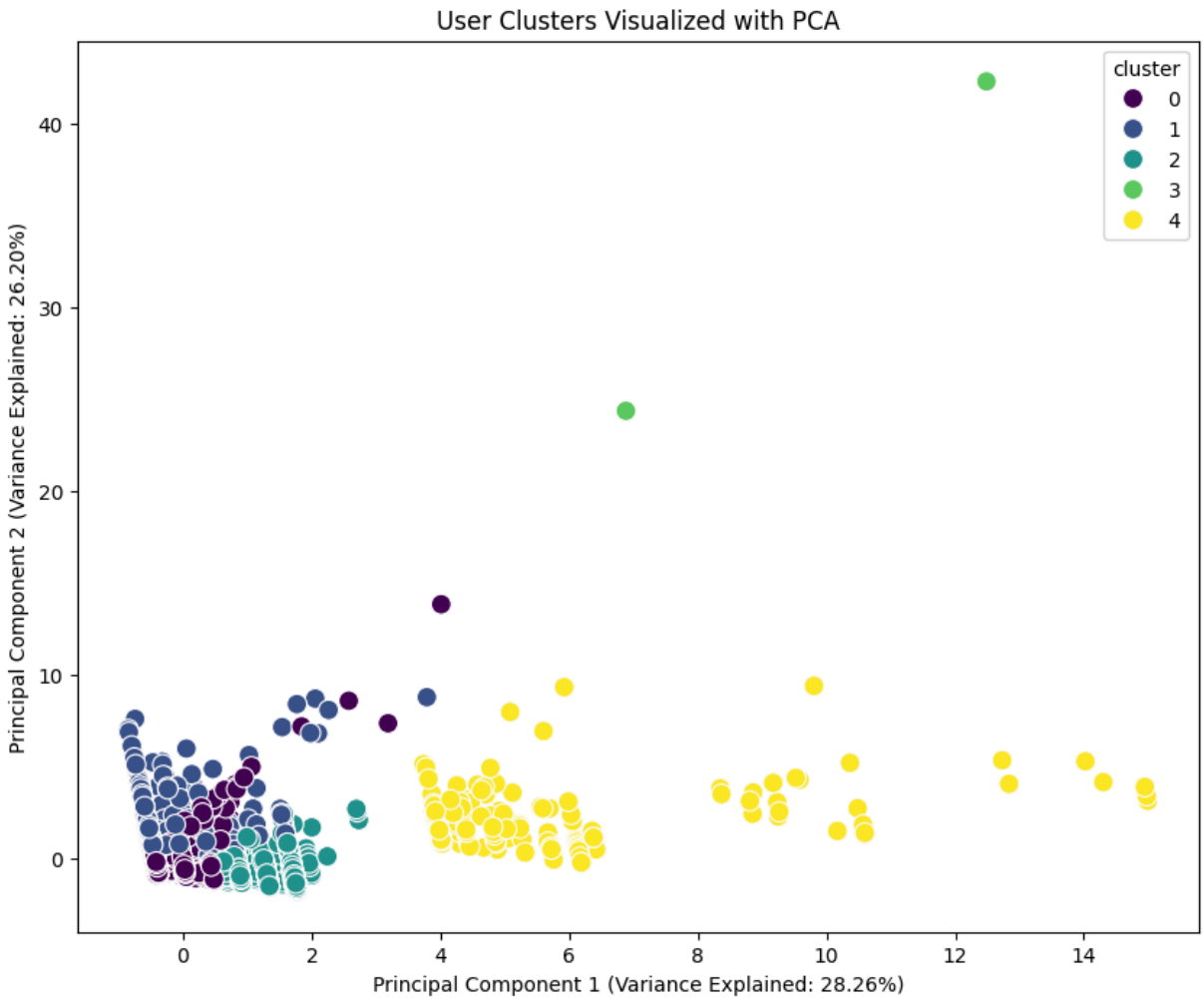
```
e.emilyClusteringAndPCA = emilyClusteringAndPCA
del emilyClusteringAndPCA
e.emilyClusteringAndPCA()
```

```
Cluster Analysis:
```

	user_reputation		reply_count		stars		review_leng	
	mean	std	mean	std	mean	std	mean	s
cluster								
0	1.378536	3.558238	0.000000	0.000000	4.843428	0.422324	146.785435	80.5592
1	1.810010	5.901049	0.000000	0.000000	4.696021	0.755776	491.624059	190.3240
2	1.255088	2.350882	0.000000	0.000000	0.383154	0.732570	161.129053	112.9716
3	400.000000	155.563492	0.000000	0.000000	5.000000	0.000000	204.833333	61.5182
4	3.661550	9.540641	1.166667	0.448525	2.641360	2.088057	256.433750	199.5711

```
<Figure size 1200x1000 with 0 Axes>
```


Pairwise Relationships Between Features by Cluster
user_reputation
reply_count
stars
review_length
cluster
0
1
2
3
4



Explanation: We aggregated the user features in order to accurately calculate the optimal k, then we performed k-clustering to determine the different types of users given our data.

- Cluster 0: This is the largest cluster of users, who are characterized by high star ratings and short reviews. They also have a low to no reply count and low reputation.
- Cluster 1: There are a decent number of users in this cluster, who are characterized by high star ratings and long reviews. They don't have much reply count and low reputation.
- Cluster 2: There are some users in this cluster, who are characterized by low star ratings and short reviews. They don't have much reply count and low reputation.
- Cluster 3: There are only 2 users in this cluser, who are characterized by a significantly high reputation and 5-star ratings. They don't have much reply count or very long reviews.
- Cluster 4: There are a handful of users in this cluster, who are characterized by a higher reply count, medium review lengths and an average star rating. They also have a low reputation.

TF-IDF & Cosine Similarity

Next we process all tokens into TF-IDF Vectors across all reviews. We also compute aggregated TF-IDF vectors where each recipe vector is a collection of all its reviews TF-IDF scores.

With the following results:
TF-IDF matrix shape: (18182, 9085)
Recipe TF-IDF matrix shape: (100, 9085)
Cosine Similarity matrix shape: (18182, 18182)
Recipe Cosine Similarity matrix shape: (100, 100)

We have 18,182 total reviews with 9085 unique tokens and then we verify the Recipe matrix with 100 recipes and the same 9085 unique tokens.
The cosine similarity matrices match up with each review having a similarity score to all other reviews and the same for our 100 recipes.

In [14]:
```
def computeTFIDF():
    vectorizer = d.TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform(d.df["clean_text"])

    d.vectorizer = vectorizer
    d.tfidf_matrix = tfidf_matrix

d.computeTFIDF = computeTFIDF
del computeTFIDF

d.computeTFIDF()
print("TF-IDF matrix shape:", d.tfidf_matrix.shape)



def computeRecipeTFIDF():
    import numpy as np

    recipe_ids = d.df["recipe_number"].unique()
    recipe_ids.sort()
    recipe_vectors = []

    for rid in recipe_ids:
        idx = d.df.index[d.df["recipe_number"] == rid].tolist()
        mat = d.tfidf_matrix[idx]
        avg_vec = mat.mean(axis=0)
        avg_vec = np.asarray(avg_vec).ravel()
        recipe_vectors.append(avg_vec)

    d.recipe_ids = recipe_ids
    d.recipe_tfidf = np.vstack(recipe_vectors)

d.computeRecipeTFIDF = computeRecipeTFIDF
del computeRecipeTFIDF
d.computeRecipeTFIDF()
print("Recipe TF-IDF matrix shape:", d.recipe_tfidf.shape)
```

```
def computeCosineSimilarity():
    d.sim_matrix = d.cosine_similarity(d.tfidf_matrix)

d.computeCosineSimilarity = computeCosineSimilarity
del computeCosineSimilarity
d.computeCosineSimilarity()
print("Cosine Similarity matrix shape:", d.sim_matrix.shape)



def computeRecipeSimilarity():
    d.recipe_sim_matrix = d.cosine_similarity(d.recipe_tfidf)

d.computeRecipeSimilarity = computeRecipeSimilarity
del computeRecipeSimilarity
d.computeRecipeSimilarity()
print("Recipe Cosine Similarity matrix shape:", d.recipe_sim_matrix.shape)
```

```
TF-IDF matrix shape: (18182, 9085)
Recipe TF-IDF matrix shape: (100, 9085)
Cosine Similarity matrix shape: (18182, 18182)
Recipe Cosine Similarity matrix shape: (100, 100)
```

What are the Top 25 and Least 25 Similar Recipes?

Next we wanted to see which are the top 25 and bottom recipes that have most similar wording. In order words, which pairs have the highest cosine similarity between them. Which we found to be the highest similar recipes to be 2 chocolate cake recipes! Moist Choclate Cake and Sandy's Chocolate Cake. We also noticed that mainly deserts have the highest similarty with some soups and one chicken recipe within our top 25 pairs. This makes sense with the wording of baking recipes require certain cooking jargon that other recipe sets might not necessarily require.

Now we take a look at our least 25 recipes, oddly enough we have a bunch of desserts again here. Perhaps even within different desserts, there exists unique jargon that causes a low similarity score. It is fascinating to see Sandy's Chocolate Cake exist both at the top of all pairs and at the lowest of all pairs with Favorite Dutch Apple Pie.

In [15]:
```
def topSimilarRecipePairs(n_recipes=100, top_k=25):
    import numpy as np

    sim = d.recipe_sim_matrix[:n_recipes, :n_recipes]

    pairs = []
    for i in range(n_recipes):
        for j in range(i + 1, n_recipes):
            pairs.append((i, j, sim[i, j]))

    pairs_sorted = sorted(pairs, key=lambda x: x[2], reverse=True)
    top_pairs = pairs_sorted[:top_k]
```

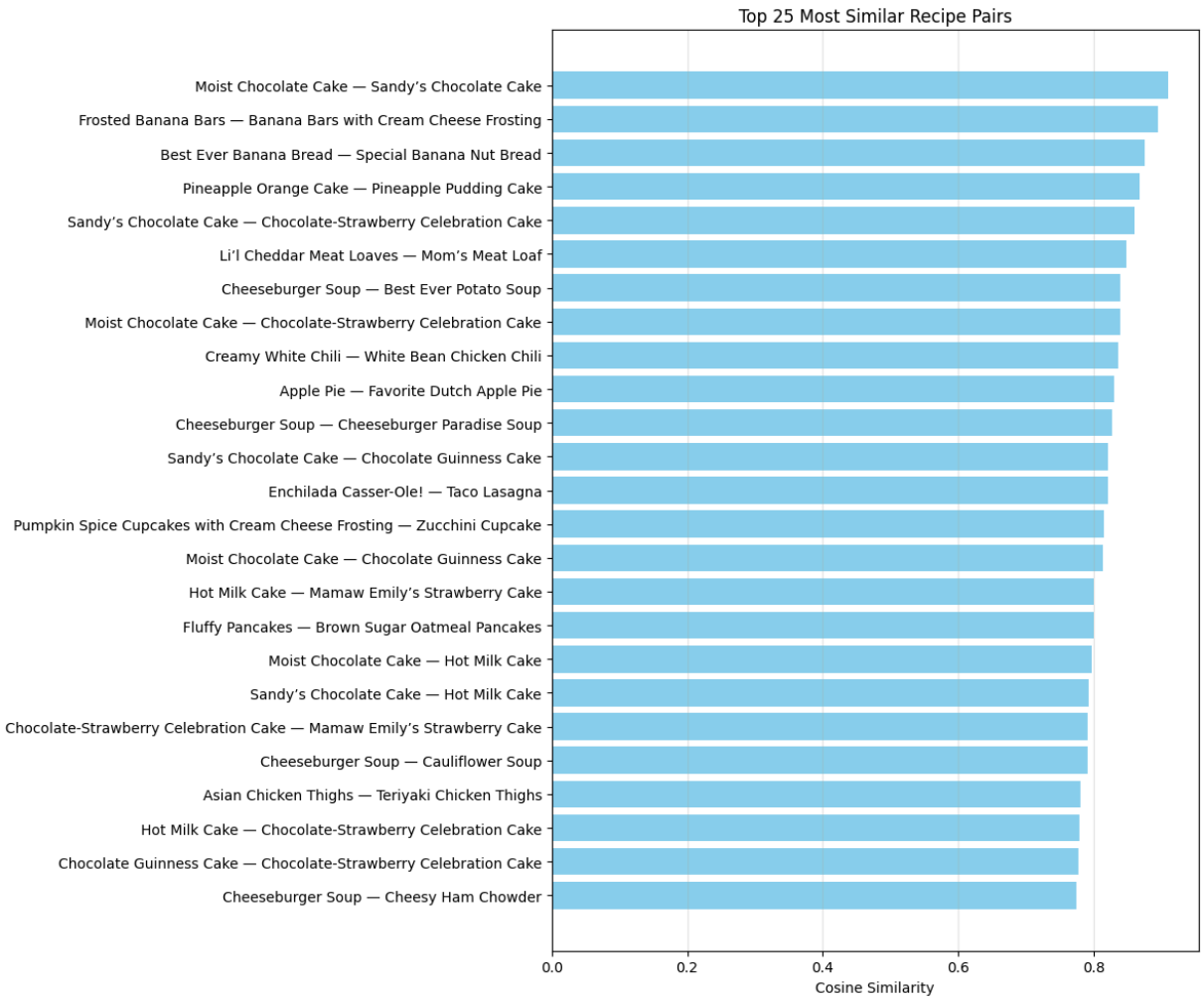
```
    results = []
    for i, j, score in top_pairs:
        rid_i = int(d.recipe_ids[i])
        rid_j = int(d.recipe_ids[j])
        results.append((rid_i, rid_j, float(score)))

    return results

d.topSimilarRecipePairs = topSimilarRecipePairs
del topSimilarRecipePairs



def lowestSimilarRecipePairs(n_recipes=100, low_k=25):
    import numpy as np

    sim = d.recipe_sim_matrix[:n_recipes, :n_recipes]

    pairs = []
    for i in range(n_recipes):
        for j in range(i + 1, n_recipes):
            pairs.append((i, j, sim[i, j]))

    pairs_sorted = sorted(pairs, key=lambda x: x[2], reverse=False)
    top_pairs = pairs_sorted[:low_k]

    results = []
    for i, j, score in top_pairs:
        rid_i = int(d.recipe_ids[i])
        rid_j = int(d.recipe_ids[j])
        results.append((rid_i, rid_j, float(score)))

    return results

d.lowestSimilarRecipePairs = lowestSimilarRecipePairs
del lowestSimilarRecipePairs
```

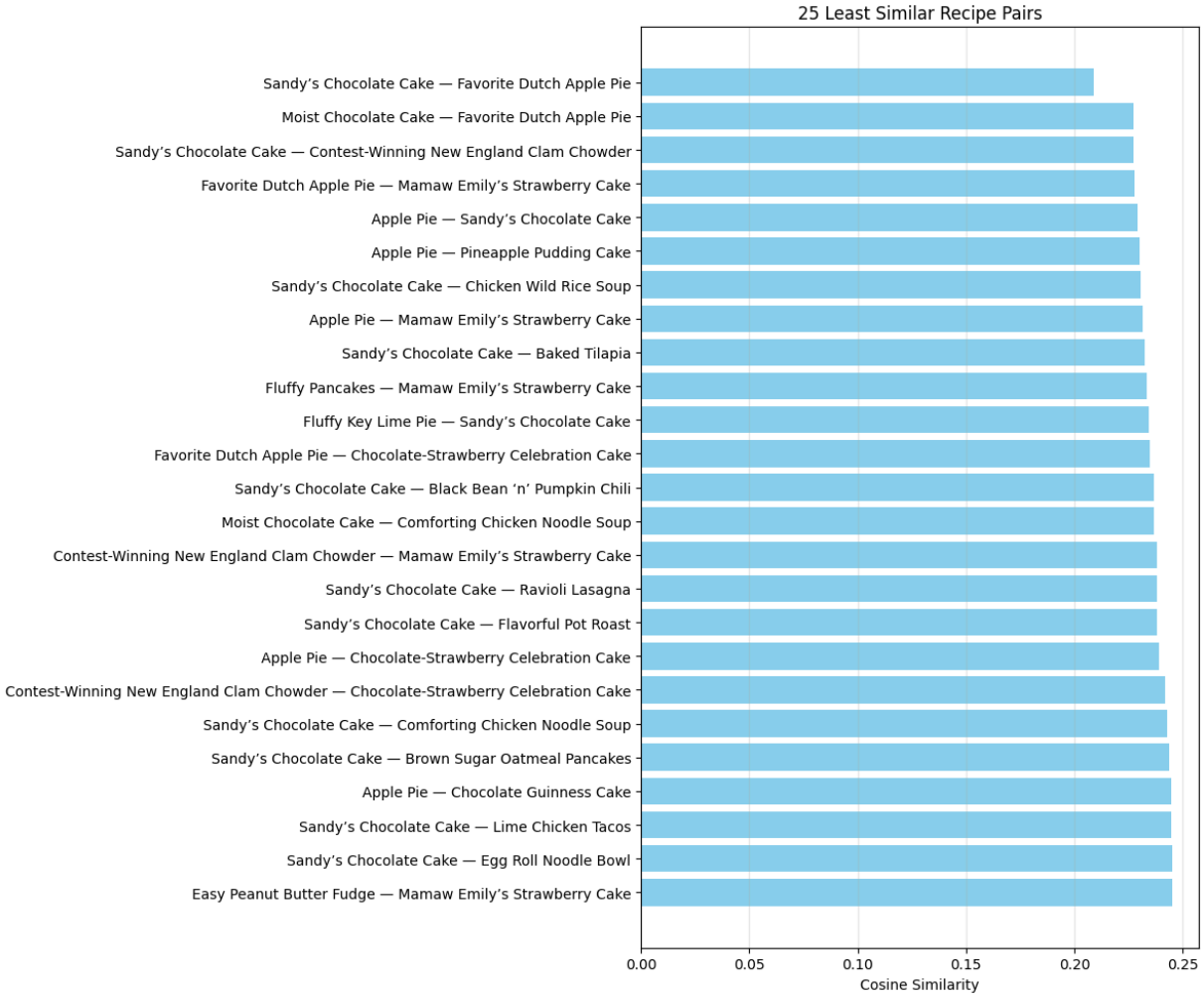
In [16]:

```
def plotRecipePairScores(pairs, title="Recipe Pair Similarities"):

    # Build ID → name mapping
    id_to_name = dict(zip(d.df["recipe_number"], d.df["recipe_name"]))

    # Unpack
    recipeA = [p[0] for p in pairs]
    recipeB = [p[1] for p in pairs]
    scores  = [p[2] for p in pairs]

    # Convert IDs → names
    labels = [f"{id_to_name.get(a, a)} — {id_to_name.get(b, b)}"
              for a, b in zip(recipeA, recipeB)]

    # Plot
    d.plt.figure(figsize=(12, 10))
    d.plt.barh(labels, scores, color="skyblue")
    d.plt.gca().invert_yaxis()
```

```
    d.plt.xlabel("Cosine Similarity")
    d.plt.title(title)
    d.plt.grid(axis="x", alpha=0.3)
    d.plt.tight_layout()
    d.plt.show()

d.plotRecipePairScores = plotRecipePairScores
del plotRecipePairScores
```

In [17]:
```
top_pairs = d.topSimilarRecipePairs(100, 25)
d.plotRecipePairScores(top_pairs, title="Top 25 Most Similar Recipe Pairs")

bottom_pairs = d.lowestSimilarRecipePairs(100, 25)
d.plotRecipePairScores(bottom_pairs, title="25 Least Similar Recipe Pairs")
```





How do recipes rank in overall Sentiment Score?

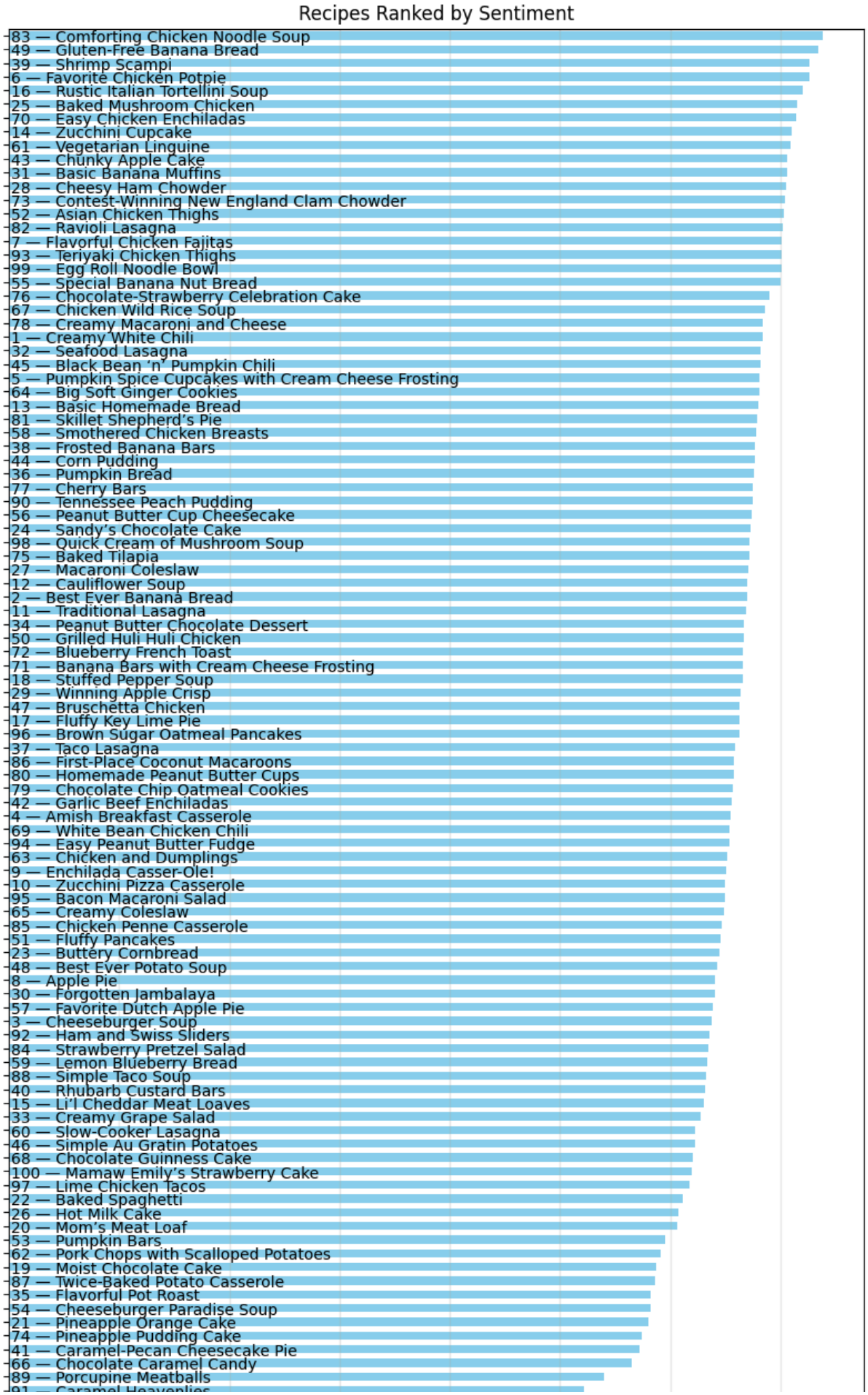
Here, we wanted to explore the overall Sentiment Score per recipe. So we made an aggregated sentiment score using NLTK VADER (Valence Aware Dictionary and sEntiment Reasoner).

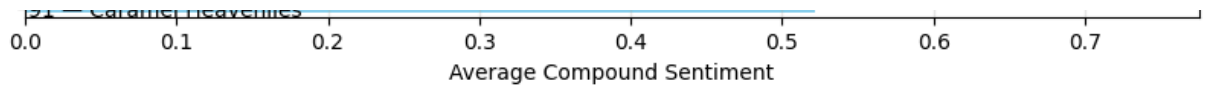
By doing so, we were able to also include the ranking of each recipe and we can visually compare the sentiment scores and where each recipe exists within the top 100 recipes in our dataset. We would expect that the number one recipe has the highest sentiment score, right? However that was not the case. Recipe number 83, Comforting Chicken Noodle Soup had the highest sentiment score. Maybe people are writing their reviews after being sick and feeling healthy and alive from such a comforting soup!

On the low end of sentiment scores the Caramel Heavenlies are sadly our bottom. Interestingly, there does seem to be a commonality of caramel recipes in the lower portion of sentiment scores.

In [18]:
```
def computeRecipeSentiment():
    d.df["sentiment_compound"] = (
        d.df["text"].fillna("").astype(str)
        .apply(lambda txt: d.vader.polarity_scores(txt)["compound"])
```

```
    )

    recipe_sent = (
        d.df[d.df["recipe_number"].isin(range(1, 101))]
        .groupby("recipe_number")["sentiment_compound"]
        .mean()
        .reset_index(name="sentiment_compound")
    )

    recipe_sent = recipe_sent.merge(
        d.df[["recipe_number", "recipe_name"]].drop_duplicates(),
        on="recipe_number",
        how="left"
    )

    d.recipe_sentiment = recipe_sent

d.computeRecipeSentiment = computeRecipeSentiment
del computeRecipeSentiment
d.computeRecipeSentiment()



def plotSortedRecipeSentiment():
    rs = d.recipe_sentiment.sort_values("sentiment_compound")
    rs["label"] = rs["recipe_number"].astype(str) + " — " + rs["recipe_name"]
    d.plt.figure(figsize=(14, 16))
    d.plt.barh(
        rs["label"],
        rs["sentiment_compound"],
        color="skyblue",
        height=0.6
    )

    ax = d.plt.gca()
    for label in ax.get_yticklabels():
        label.set_ha('left')
        label.set_x(0.015)

    ax.set_ylim(-0.5, len(rs) - 0.5)

    d.plt.xlabel("Average Compound Sentiment")
    d.plt.title("Recipes Ranked by Sentiment")
    d.plt.grid(axis="x", alpha=0.3)
    d.plt.subplots_adjust(left=0.35)
    d.plt.show()

d.plotSortedRecipeSentiment = plotSortedRecipeSentiment
del plotSortedRecipeSentiment
d.plotSortedRecipeSentiment()
```


Recipes Ranked by Sentiment
83 — Comforting Chicken Noodle Soup
49 — Gluten-Free Banana Bread
39 — Shrimp Scampi
6 — Favorite Chicken Potpie
16 — Rustic Italian Tortellini Soup
25 — Baked Mushroom Chicken
70 — Easy Chicken Enchiladas
14 — Zucchini Cupcake
61 — Vegetarian Linguine
43 — Chunky Apple Cake
31 — Basic Banana Muffins
28 — Cheesy Ham Chowder
73 — Contest-Winning New England Clam Chowder
52 — Asian Chicken Thighs
82 — Ravioli Lasagna
7 — Flavorful Chicken Fajitas
93 — Teriyaki Chicken Thighs
99 — Egg Roll Noodle Bowl
55 — Special Banana Nut Bread
76 — Chocolate-Strawberry Celebration Cake
67 — Chicken Wild Rice Soup
78 — Creamy Macaroni and Cheese
1 — Creamy White Chili
32 — Seafood Lasagna
45 — Black Bean 'n' Pumpkin Chili
5 — Pumpkin Spice Cupcakes with Cream Cheese Frosting
64 — Big Soft Ginger Cookies
13 — Basic Homemade Bread
81 — Skillet Shepherd's Pie
58 — Smothered Chicken Breasts
38 — Frosted Banana Bars
44 — Corn Pudding
36 — Pumpkin Bread
77 — Cherry Bars
90 — Tennessee Peach Pudding
56 — Peanut Butter Cup Cheesecake
24 — Sandy's Chocolate Cake
98 — Quick Cream of Mushroom Soup
75 — Baked Tilapia
27 — Macaroni Coleslaw
12 — Cauliflower Soup
2 — Best Ever Banana Bread
11 — Traditional Lasagna
34 — Peanut Butter Chocolate Dessert
50 — Grilled Huli Huli Chicken
72 — Blueberry French Toast
71 — Banana Bars with Cream Cheese Frosting
18 — Stuffed Pepper Soup
29 — Winning Apple Crisp
47 — Bruschetta Chicken
17 — Fluffy Key Lime Pie
96 — Brown Sugar Oatmeal Pancakes
37 — Taco Lasagna
86 — First-Place Coconut Macaroons
80 — Homemade Peanut Butter Cups
79 — Chocolate Chip Oatmeal Cookies
42 — Garlic Beef Enchiladas
4 — Amish Breakfast Casserole
69 — White Bean Chicken Chili
94 — Easy Peanut Butter Fudge
63 — Chicken and Dumplings
9 — Enchilada Casser-Ole!
10 — Zucchini Pizza Casserole
95 — Bacon Macaroni Salad
65 — Creamy Coleslaw
85 — Chicken Penne Casserole
51 — Fluffy Pancakes
23 — Buttery Cornbread
48 — Best Ever Potato Soup
8 — Apple Pie
30 — Forgotten Jambalaya
57 — Favorite Dutch Apple Pie
3 — Cheeseburger Soup
92 — Ham and Swiss Sliders
84 — Strawberry Pretzel Salad
59 — Lemon Blueberry Bread
88 — Simple Taco Soup
40 — Rhubarb Custard Bars
15 — Li'l Cheddar Meat Loaves
33 — Creamy Grape Salad
60 — Slow-Cooker Lasagna
46 — Simple Au Gratin Potatoes
68 — Chocolate Guinness Cake
100 — Mamaw Emily's Strawberry Cake
97 — Lime Chicken Tacos
22 — Baked Spaghetti
26 — Hot Milk Cake
20 — Mom's Meat Loaf
53 — Pumpkin Bars
62 — Pork Chops with Scalloped Potatoes
19 — Moist Chocolate Cake
87 — Twice-Baked Potato Casserole
35 — Flavorful Pot Roast
54 — Cheeseburger Paradise Soup
21 — Pineapple Orange Cake
74 — Pineapple Pudding Cake
41 — Caramel-Pecan Cheesecake Pie
66 — Chocolate Caramel Candy
89 — Porcupine Meatballs



Multinomial Naive Bayes

The goal of this analytical technique is to attempt to build a positive/negative review classifier using the Multinomial Naive Bayes algorithm. We will use the TF-IDF vectors initialized by Danny as our features and the star ratings as our labels. We will evaluate the performance of our model using accuracy, macro F1 score, negative recall, and negative precision.

> Note that this model was originally intended to predict specific star ratings, at least at the resolution of 1 star. However, upon further analysis during development, there exists not enough data nor sophistication in MNBayes to predict at that resolution. Hence, this classifier focuses on the binary classification between the bins of 0-2 stars (negative reviews) and 3-5 stars (positive reviews). This is a common approach in sentiment analysis when the data does not support finer-grained classification.

In [19]:
```
def kianThisBlock():
    # Imports
    k.trainTestSplit    = __import__('sklearn.model_selection', fromlist=['train_te
    k.multinomialNB       = __import__('sklearn.naive_bayes', fromlist=['Multinomia
    k.classificationReport = __import__('sklearn.metrics', fromlist=['classificatio
    k.confusionMatrix      = __import__('sklearn.metrics', fromlist=['confusion_mat
    k.accuracyScore        = __import__('sklearn.metrics', fromlist=['accuracy_scor
    k.f1Score              = __import__('sklearn.metrics', fromlist=['f1_score']).f

    k.np  = __import__('numpy')
    k.pd  = __import__('pandas')
    k.plt = __import__("matplotlib.pyplot", fromlist=["pyplot"])
    k.sns = __import__('seaborn')

    # Config
    k.kianMNBayesConfig = {
        "TEST_SIZE":    0.2,
        "RANDOM_STATE": 42,

                # This was regrettably what I had to do to get a somewhat balanced
        "LABEL_MAP":    {
            0: "neg",
            1: "neg",
            2: "neg",
            3: "pos",
            4: "pos",
            5: "pos"
        },
    }

    # Data loading + binning
```

```
k.df = d.df[["stars", "clean_text"]].dropna(subset=["clean_text"]).copy()
k.df["label"] = k.df["stars"].map(k.kianMNBayesConfig["LABEL_MAP"])

print(f"Shape: {k.df.shape}")
print(f"\nOriginal star distribution:\n{k.df['stars'].value_counts().sort_index
print(f"\nBinned label distribution:\n{k.df['label'].value_counts()}")

# Train/test split + oversampling
k.x = k.df["clean_text"]
k.y = k.df["label"]

k.xTrain, k.xTest, k.yTrain, k.yTest = k.trainTestSplit(
    k.x, k.y,
    test_size=k.kianMNBayesConfig["TEST_SIZE"],
    random_state=k.kianMNBayesConfig["RANDOM_STATE"],
    stratify=k.y
)

k.negMask = k.yTrain == "neg"
k.negX = k.xTrain[k.negMask]
k.negY = k.yTrain[k.negMask]
k.nCopies = (k.yTrain == "pos").sum() // k.negMask.sum()

k.xTrainBal = k.pd.concat([k.xTrain] + [k.negX] * (k.nCopies - 1), ignore_index
k.yTrainBal = k.pd.concat([k.yTrain] + [k.negY] * (k.nCopies - 1), ignore_index

print(f"\nTrain (original): {len(k.xTrain)}  |  Test: {len(k.xTest)}")
print(f"Train (oversampled): {len(k.xTrainBal)}")
print(f"Oversampled balance:\n{k.yTrainBal.value_counts()}")

# Vectorize (bigrams + pruning)
k.vectorizer = d.TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=3,
    max_df=0.9,
)

k.xTrainVec = k.vectorizer.fit_transform(k.xTrainBal)
k.xTestVec  = k.vectorizer.transform(k.xTest)

print(f"\nVocabulary size: {len(k.vectorizer.vocabulary_):,}")
print(f"Train matrix:    {k.xTrainVec.shape}")
print(f"Test matrix:     {k.xTestVec.shape}")

# Alpha grid search
k.alphas = [0.01, 0.05, 0.1, 0.25, 0.5, 1.0]
k.gridResults = []

for k._a in k.alphas:
    k._m = k.multinomialNB(alpha=k._a, class_prior=[0.5, 0.5])
    k._m.fit(k.xTrainVec, k.yTrainBal)
    k._preds = k._m.predict(k.xTestVec)
    k._acc   = k.accuracyScore(k.yTest, k._preds)
    k._macro = k.f1Score(k.yTest, k._preds, average='macro')
    k._negR = k.classificationReport(k.yTest, k._preds, output_dict=True)['neg'
    k.gridResults.append((k._a, k._acc, k._macro, k._negR))
```

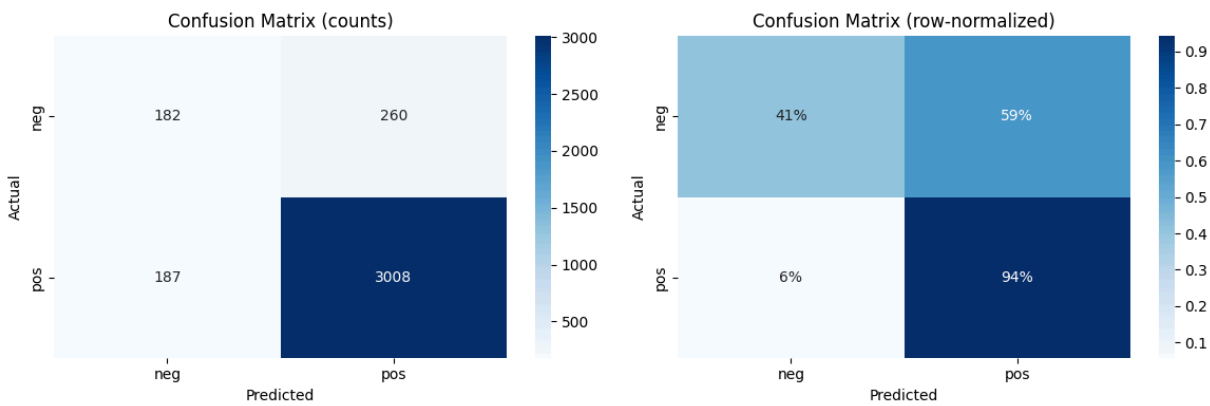
```
        print(f"alpha={k._a:<5}  acc={k._acc:.4f}  macro_f1={k._macro:.4f}  neg_rec

    k.bestAlpha = max(k.gridResults, key=lambda x: x[2])[0]
    print(f"\nBest alpha by macro F1: {k.bestAlpha}")

    k.model = k.multinomialNB(alpha=k.bestAlpha, class_prior=[0.5, 0.5])
    k.model.fit(k.xTrainVec, k.yTrainBal)
    k.yPred = k.model.predict(k.xTestVec)

    print(f"\n--- Final model (alpha={k.bestAlpha}) ---")
    print(f"Accuracy: {k.accuracyScore(k.yTest, k.yPred):.4f}\n")
    print(k.classificationReport(k.yTest, k.yPred))
    print(k.confusionMatrix(k.yTest, k.yPred, labels=["neg", "pos"]))

    # Threshold tuning
    k.yProba = k.model.predict_proba(k.xTestVec)
    k.negIdx = list(k.model.classes_).index("neg")

    k.thresholds = [0.50, 0.45, 0.40, 0.35, 0.30, 0.25, 0.20]
    k.threshResults = []

    print(f"\n{'thresh':<8} {'acc':>6} {'macro_f1':>8} {'neg_P':>6} {'neg_R':>6} {'
    print("-" * 52)
    for k._t in k.thresholds:
        k._preds = k.np.where(k.yProba[:, k.negIdx] >= k._t, "neg", "pos")
        k._acc   = k.accuracyScore(k.yTest, k._preds)
        k._macro = k.f1Score(k.yTest, k._preds, average='macro')
        k._rd    = k.classificationReport(k.yTest, k._preds, output_dict=True)
        k._negP, k._negR = k._rd['neg']['precision'], k._rd['neg']['recall']
        k._posP, k._posR = k._rd['pos']['precision'], k._rd['pos']['recall']
        k.threshResults.append((k._t, k._acc, k._macro, k._negP, k._negR, k._posP,
        print(f"{k._t:<8.2f} {k._acc:>6.4f} {k._macro:>8.4f} {k._negP:>6.2f} {k._ne

    k.bestThresh = max(k.threshResults, key=lambda x: x[2])
    print(f"\nBest threshold by macro F1: {k.bestThresh[0]:.2f}  "
          f"(acc={k.bestThresh[1]:.4f}, macro_f1={k.bestThresh[2]:.4f})")

    k.yPredTuned = k.np.where(
        k.yProba[:, k.negIdx] >= k.bestThresh[0], "neg", "pos"
    )

    print(f"\n--- Final (alpha={k.bestAlpha}, threshold={k.bestThresh[0]}) ---")
    print(f"Accuracy: {k.accuracyScore(k.yTest, k.yPredTuned):.4f}\n")
    print(k.classificationReport(k.yTest, k.yPredTuned))
    print(k.confusionMatrix(k.yTest, k.yPredTuned, labels=["neg", "pos"]))

    # Confusion matrix heatmap, I love my heatmaps <3
    k.labels = ["neg", "pos"]

    k.figCm, k.axesCm = k.plt.subplots(1, 2, figsize=(12, 4))

    k.cm = k.confusionMatrix(k.yTest, k.yPredTuned, labels=k.labels)
    k.sns.heatmap(k.cm, annot=True, fmt='d', cmap='Blues',
                  xticklabels=k.labels, yticklabels=k.labels, ax=k.axesCm[0])
    k.axesCm[0].set_xlabel('Predicted')
    k.axesCm[0].set_ylabel('Actual')
```

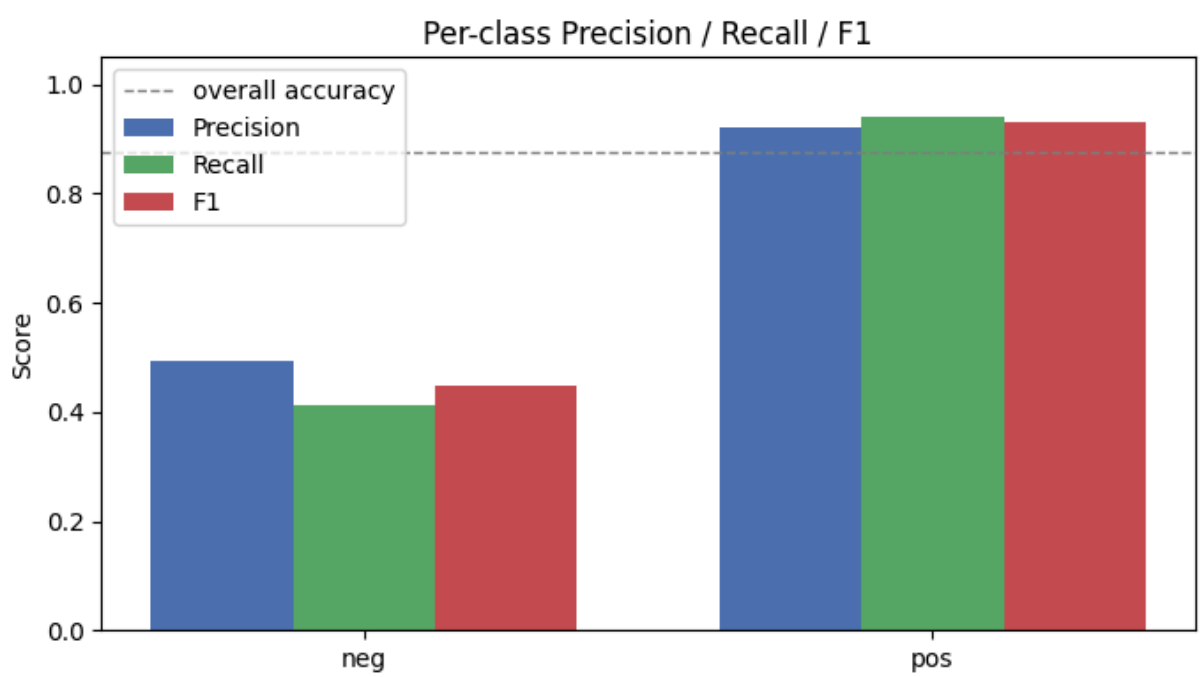
```
k.axesCm[0].set_title('Confusion Matrix (counts)')

k.cmNorm = k.cm.astype(float) / k.cm.sum(axis=1, keepdims=True)
k.sns.heatmap(k.cmNorm, annot=True, fmt='.0%', cmap='Blues',
              xticklabels=k.labels, yticklabels=k.labels, ax=k.axesCm[1])
k.axesCm[1].set_xlabel('Predicted')
k.axesCm[1].set_ylabel('Actual')
k.axesCm[1].set_title('Confusion Matrix (row-normalized)')

k.plt.tight_layout()
k.plt.show()

# Per-class precision / recall / F1 bar chart
k.reportDict = k.classificationReport(k.yTest, k.yPredTuned, output_dict=True)

k.precisions = [k.reportDict[c]['precision'] for c in k.labels]
k.recalls    = [k.reportDict[c]['recall']    for c in k.labels]
k.f1s        = [k.reportDict[c]['f1-score']  for c in k.labels]

k._x = k.np.arange(len(k.labels))
k._w = 0.25

k.figPrf, k.axPrf = k.plt.subplots(figsize=(7, 4))
k.axPrf.bar(k._x - k._w, k.precisions, k._w, label='Precision', color='#4c72b0'
k.axPrf.bar(k._x,         k.recalls,    k._w, label='Recall',    color='#55a868
k.axPrf.bar(k._x + k._w,  k.f1s,        k._w, label='F1',        color='#c44e52

k.axPrf.set_xticks(k._x)
k.axPrf.set_xticklabels(k.labels)
k.axPrf.set_ylabel('Score')
k.axPrf.set_title('Per-class Precision / Recall / F1')
k.axPrf.set_ylim(0, 1.05)
k.axPrf.axhline(y=k.accuracyScore(k.yTest, k.yPredTuned), color='grey',
                linestyle='--', linewidth=1, label='overall accuracy')
k.axPrf.legend()
k.plt.tight_layout()
k.plt.show()

# Training distribution vs. prediction distribution
k.figDist, k.axesDist = k.plt.subplots(1, 2, figsize=(10, 4), sharey=False)

k.pd.Series(k.yTrainBal).value_counts().reindex(k.labels).plot.bar(
    ax=k.axesDist[0])
k.axesDist[0].set_title('Training set (oversampled)')
k.axesDist[0].set_xlabel('Label')
k.axesDist[0].set_ylabel('Count')

k.pd.Series(k.yPredTuned).value_counts().reindex(k.labels, fill_value=0).plot.b
    ax=k.axesDist[1])
k.axesDist[1].set_title('Test set // predicted (threshold-tuned)')
k.axesDist[1].set_xlabel('Label')
k.axesDist[1].set_ylabel('Count')

k.plt.tight_layout()
k.plt.show()
```

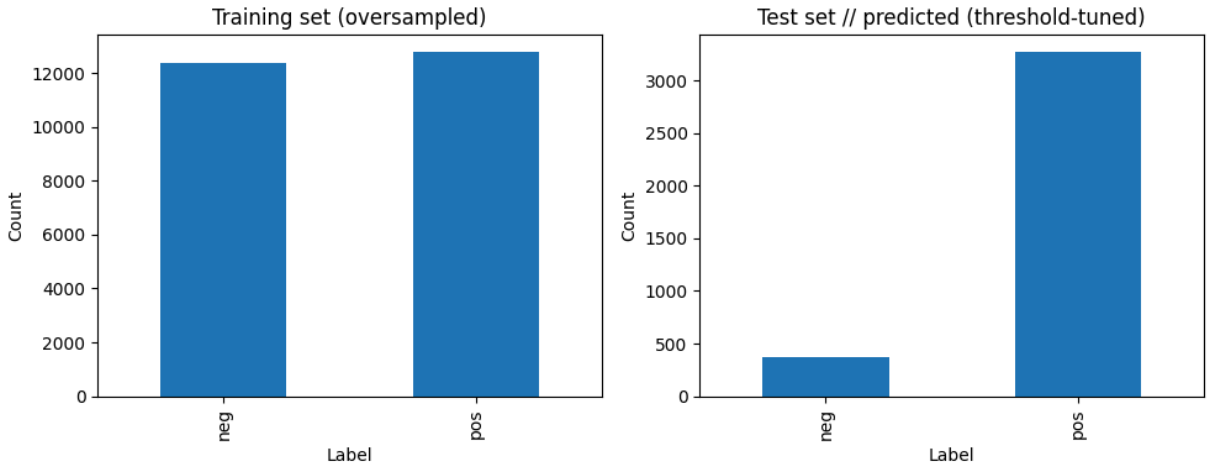
```
kianThisBlock()
del kianThisBlock
```

```
Shape: (18182, 3)

Original star distribution:
stars
0     1696
1      280
2      232
3      490
4     1655
5    13829
Name: count, dtype: int64

Binned label distribution:
label
pos    15974
neg     2208
Name: count, dtype: int64

Train (original): 14545  |  Test: 3637
Train (oversampled): 25141
Oversampled balance:
label
pos    12779
neg    12362
Name: count, dtype: int64

Vocabulary size: 33,752
Train matrix:    (25141, 33752)
Test matrix:     (3637, 33752)
alpha=0.01   acc=0.8787  macro_f1=0.6409  neg_recall=0.2670
alpha=0.05   acc=0.8807  macro_f1=0.6480  neg_recall=0.2783
alpha=0.1    acc=0.8820  macro_f1=0.6534  neg_recall=0.2873
alpha=0.25   acc=0.8842  macro_f1=0.6608  neg_recall=0.2986
alpha=0.5    acc=0.8837  macro_f1=0.6537  neg_recall=0.2828
alpha=1.0    acc=0.8875  macro_f1=0.6643  neg_recall=0.2964

Best alpha by macro F1: 1.0

--- Final model (alpha=1.0) ---
Accuracy: 0.8875

              precision    recall  f1-score   support

         neg       0.57      0.30      0.39       442
         pos       0.91      0.97      0.94      3195

    accuracy                           0.89      3637
   macro avg       0.74      0.63      0.66      3637
weighted avg       0.87      0.89      0.87      3637

[[ 131  311]
 [  98 3097]]

thresh      acc macro_f1  neg_P  neg_R  pos_P  pos_R
-----------------------------------------------------
0.50     0.8875   0.6643   0.57   0.30   0.91   0.97
```

```
0.45     0.8812   0.6703   0.52   0.33   0.91   0.96
0.40     0.8771   0.6898   0.49   0.41   0.92   0.94
0.35     0.8592   0.6826   0.43   0.47   0.93   0.91
0.30     0.8339   0.6716   0.37   0.54   0.93   0.87
0.25     0.7957   0.6445   0.32   0.59   0.94   0.82
0.20     0.7322   0.6015   0.26   0.66   0.94   0.74

Best threshold by macro F1: 0.40  (acc=0.8771, macro_f1=0.6898)

--- Final (alpha=1.0, threshold=0.4) ---
Accuracy: 0.8771

              precision    recall  f1-score   support

         neg       0.49      0.41      0.45       442
         pos       0.92      0.94      0.93      3195

    accuracy                           0.88      3637
   macro avg       0.71      0.68      0.69      3637
weighted avg       0.87      0.88      0.87      3637

[[ 182  260]
 [ 187 3008]]
```







Explanation: The MNBayes model began simple and poor-performing, specifically in that its configuration to classify at a 1-star resolution was simply asking too much of both the model architecture and the training data, which skews heavily towards the positive and specifically 5 stars, as seen above. By binning the star ratings into negative (0-2) and positive (3-5), we simplified the classification task to a binary problem, classifying reviews as either negative or positive. Multiple other optimizations were applied to improve model performance, including:

- Switching from CountVectorizer to Danny's TF-IDF vectors, which better capture the importance of words in reviews.
- Adjusting the training data pool so that the model doesn't so aggressively assume a review will be positive
- Adding bigram features to better capture concepts like "not good" or "very bad"
- Manually adjusting the classification threshold to make it more willing to classify as negative, as overclassification as positive remains a problem

In the end, the overclassification problem was not fully solved, and we believe we have reached near the limit of what MNBayes can do with this dataset. The model performs decently at classifying positive reviews, but struggles to identify negative reviews, a common issue in imbalanced datasets.

Conclusion

This project investigated what recipe and user characteristics influence review outcomes on a recipe platform, focusing on engagement metrics like thumbs up, reply count, and star ratings across 18,182 reviews and 100 recipes.

Hypothesis Results

- **H1 — Not Supported:** We hypothesized that veteran reviewers would write longer, more nuanced reviews. However, the Spearman correlation between user reputation

and review length was only ρ = 0.037, which is statistically significant but practically negligible.

- **H2 — Partially Supported:** We hypothesized that extreme reviews attract more engagement. The data shows that low-star reviews do receive more replies (ρ = -0.121) and slightly more thumbs up, but the effect is weak overall.

Key Findings

- **Review length and reply count** were the strongest predictors of thumbs up (ρ = 0.181 and ρ = 0.177 respectively), suggesting that substance and controversy drive engagement more than user credibility.
- **User reputation** had virtually no practical relationship with any outcome measured, despite being statistically significant across all pairs tested.
- **K-Means clustering** revealed that the majority of users fall into a passive reviewer profile — short reviews, high star ratings, and little to no engagement.
- **TF-IDF and sentiment analysis** showed that review language is overwhelmingly positive across all recipes, with words like "make", "love", and "delicious" dominating the corpus.
- **Cosine similarity** identified that recipe pairs of the same dish type (e.g., two chocolate cake recipes) naturally share the most similar review language.

Overall Takeaway

User behavior on this platform is largely passive. Most reviews have zero thumbs up, zero replies, and a reputation score of 1, which severely limits the detectable range of behavioral differences. While all 7 Spearman correlations were statistically significant, none were practically strong. On platforms like this, **what you say matters more than who you are** — a longer, more critical review from a new user will outperform a short five-star review from a seasoned one.

References:

https://realpython.com/python-nltk-sentiment-analysis/#getting-started-with-nltk
https://www.nltk.org/howto/semantics.html
https://github.com/amueller/word_cloud